Manual of plotastrodata

Yusuke Aso

Contents

1 Quick Start **3**
- 1.1 Make a 2D Map from a FITS File . . . 3
- 1.2 Make Channel Maps from a Cube . . . 3
- 1.3 Use NumPy Arrays Directly . . . 3
- 1.4 Common Things to Check . . . 4

2 Recipes **4**
- 2.1 Overlay Multiple Data Sets . . . 4
- 2.2 Plot a Position–Velocity Diagram . . . 5
- 2.3 Make an RGB Image . . . 5
- 2.4 Extract Spectra and Slices . . . 5
- 2.5 Modify Data and Write FITS . . . 6

3 Data Model **7**
- 3.1 Runtime Types . . . 7
- 3.2 Integer Counts and Validation . . . 7
- 3.3 Zero-Dimensional Arrays . . . 7
- 3.4 Finite Real Scalar Inputs . . . 8
- 3.5 Basic Array Convention . . . 8
- 3.6 Spatial Coordinates . . . 9
- 3.7 Velocity Coordinates . . . 9
- 3.8 FITS Input and Array Input . . . 9
- 3.9 The `AstroData` Object . . . 10
- 3.10 Swapped Axes and Overlay Angles . . . 10
- 3.11 Noise and Units . . . 11
- 3.12 Position–Velocity Diagrams . . . 11
- 3.13 Multiple Layers . . . 11

4 Data Input **11**
- 4.1 Noise measurement . . . 13

5 Analysis Methods **15**
- 5.1 FITS Conversion and Cached Data . . . 18

6 Plotting Methods **20**
6.1 Core Class Parameters . . . 20
6.2 Validated Configuration . . . 22
6.3 Arguments for the methods to make maps . . . 22
6.4 Maps . . . 23
6.5 Patches, markers, text, lines, arrows, and scale bar . . . 25
6.6 Setting axes and saving figures . . . 26
6.7 Animation . . . 29
6.8 Other plotting functions . . . 29

Appendix **31**

A Coordinate Transformations **31**

B Line-of-Sight Rotation **31**

C Fourier Transform **34**

D Fitting **36**
D.1 Grid Search . . . 39

E Additional Utilities **39**

F Estimating Plotting Ranges **41**

G Simple 2D Script Generation **42**

H Cube Script Generation **42**

1 Quick Start

This section gives the shortest path from a FITS file or a set of NumPy arrays to a useful figure. The later sections give the full reference for the same commands.

1.1 Make a 2D Map from a FITS File

```
from plotastrodata.plot_utils import PlotAstroData

p = PlotAstroData(fitsimage='image.fits',
                  center='00h00m00s 00d00m00s',
                  rmax=3.0)
p.add_color(fitsimage='image.fits',
            stretch='linear',
            show_cbar=True,
            cblabel=r'Jy beam$^{-1}$')
p.add_contour(fitsimage='image.fits',
              levels=[3, 6, 12],
              colors='white',
              show_beam=False)
p.add_scalebar(length=50 / 140, label='50 au')
p.set_axis()
p.savefig('image.png')
```

The plotting frame is defined by `PlotAstroData`. The data layers are then added by `add_color`, `add_contour`, `add_segment`, or `add_rgb`. Finally, `set_axis` formats the axes and `savefig` writes the image file.

1.2 Make Channel Maps from a Cube

```
from plotastrodata.plot_utils import PlotAstroData

p = PlotAstroData(fitsimage='cube.fits',
                  rmax=5.0,
                  vmin=-5.0,
                  vmax=5.0,
                  nrows=4,
                  ncols=5)
p.add_color(fitsimage='cube.fits', stretch='linear')
p.add_contour(fitsimage='cube.fits',
              levels=[3, 6, 12],
              colors='black',
              show_beam=False)
p.set_axis()
p.savefig('cube.png')
```

When the selected velocity range contains multiple channels, `PlotAstroData` makes channel maps. If the figure spans multiple pages, `savefig('cube.png')` writes files such as `cube_0.png`, `cube_1.png`, and so on.

1.3 Use NumPy Arrays Directly

```
from plotastrodata.plot_utils import PlotAstroData

p = PlotAstroData(v=v, rmax=5.0, vmin=-5.0, vmax=5.0)
p.add_color(data=data,
            x=x,
            y=y,
            v=v,
            beam=[bmaj, bmin, bpa],
            sigma=sigma,
            bunit=r'Jy beam$^{-1}$')
p.set_axis()
p.savefig('array_input.png')
```

For array input, the data shape should be `(ny, nx)` for a 2D image or `(nv, ny, nx)` for a cube. The arrays `x`, `y`, and `v` are one-dimensional coordinate grids.

1.4 Common Things to Check

- Use `rmax`, or explicit `xmin`, `xmax`, `ymin`, and `ymax`, to define the spatial plotting range.
- Use `vmin`, `vmax`, and `vsys` in km s^{-1} for cubes and PV diagrams.
- Use `center='common'` for overlay layers that should use the plotting-frame center.
- Give `dist` only when spatial coordinates and beams should be scaled from arcsec to a physical unit such as au.
- Call `set_axis()` or `set_axis_radec()` before `savefig()`.

2 Recipes

The recipes below are intentionally short. They show the command pattern for common tasks; the reference sections after them describe the arguments in detail.

2.1 Overlay Multiple Data Sets

```
from plotastrodata.plot_utils import PlotAstroData

p = PlotAstroData(fitsimage='continuum.fits', rmax=2.0)
p.add_color(fitsimage='continuum.fits',
            stretch='asinh',
            stretchscale=0.02)
p.add_contour(fitsimage='line_mom0.fits',
              levels=[3, 6, 12, 24],
              center='common',
              colors='black',
              show_beam=False)
p.set_axis()
p.savefig('overlay.png')
```

Use `center='common'` when the overlay should be registered to the center of the plotting frame. This is the usual choice for comparing different FITS files of the same target.

2.2 Plot a Position–Velocity Diagram

```
from plotastrodata.plot_utils import PlotAstroData

p = PlotAstroData(fitsimage='pv.fits',
                  pv=True,
                  rmax=5.0,
                  vmin=-4.0,
                  vmax=4.0)
p.add_color(fitsimage='pv.fits', pv=True)
p.add_contour(fitsimage='pv.fits',
              pv=True,
              levels=[3, 6, 12],
              colors='black',
              show_beam=False)
p.set_axis()
p.savefig('pv.png')
```

In PV mode, the horizontal axis is the spatial offset and the vertical axis is velocity. Region, line, arrow, segment, and scalebar overlays are rejected in this mode because the two axes have different physical units.

2.3 Make an RGB Image

```
from plotastrodata.plot_utils import PlotAstroData

p = PlotAstroData(fitsimage='red.fits')
p.add_rgb(fitsimage=['red.fits', 'green.fits', 'blue.fits'],
          stretch=['linear', 'log', 'linear'],
          center='common')
p.set_axis()
p.savefig('rgb.png')
```

The red, green, and blue inputs must have compatible shapes after they are read and trimmed into the plotting frame.

2.4 Extract Spectra and Slices

```
from plotastrodata.analysis_utils import AstroData, AstroFrame

d = AstroData(fitsimage='cube.fits')
f = AstroFrame(fitsimage='cube.fits', vmin=-5.0, vmax=5.0)
f.read(d)

v, profiles, fits = d.profile(xlist=[0.0], ylist=[0.0])
r, pv = d.slice(length=4.0, pa=45.0)
```

`profile` returns spectra at selected positions. `slice` returns a 1D cut for a 2D image or a PV diagram for a 3D cube.

2.5 Modify Data and Write FITS

```
from plotastrodata.analysis_utils import AstroData, AstroFrame

d = AstroData(fitsimage='cube.fits')
f = AstroFrame(fitsimage='cube.fits', vmin=-5.0, vmax=5.0)
f.read(d)

d.binning(width=[2, 1, 1])
d.mask(includepix=[3 * d.sigma, 1e99])
d.writetofits(fitsimage='cube_masked.fits')
```

The output FITS file reuses the original header when possible and updates the coordinate keywords affected by the analysis methods.

3 Data Model

`plotastrodata` represents astronomical images and cubes using a small set of arrays and metadata. The most important object is the data array itself, together with one-dimensional coordinate grids for the horizontal, vertical, and velocity axes. The package can read these quantities from a FITS file, or they can be supplied directly as NumPy arrays.

3.1 Runtime Types

Computed scalar outputs use Python `float`, `int`, and `bool`. Extended-precision NumPy floating scalars, such as `np.longdouble`, are preserved. NumPy arrays, including zero-dimensional arrays, remain arrays; scalar normalization does not change their dtypes. Numerical operations such as temperature conversion may themselves promote a dtype. Generic container and pass-through helpers do not recursively convert user values. Integer constants remain integers. The rotation-matrix function `matrix_utils.Mrot3d` returns a NumPy array.

3.2 Integer Counts and Validation

Count arguments checked as integers accept Python `int` and NumPy integer scalars. Python/NumPy booleans, floats (including `2.0`), NaN, infinity, and arrays (including zero-dimensional arrays) are rejected with `ValueError`; validation does not round or convert them.

Positive counts include interpolation factors, bin widths, spatial skips, channel-map rows and columns, velocity skips, minor-tick subdivisions, `normalize` histogram bins, and MCMC steps, temperatures, trials, and core counts. Zero is allowed for `veldigit`, MCMC `nburnin`, and automatic `plotprofile(nrows=0)`. Both `nwalkersperdim` and inverse-real-FFT `nx` must be at least two. Additionally, `nburnin < nsteps`, and `nx` must match the half-spectrum length. Walker counts below two raise an error rather than being replaced by two. Binning may still reduce oversized valid widths to the available axis sizes. Explicit edges override unused `noise_utils.normalize` bin counts.

These rules apply to count arguments, not every numerical parameter. Signed channel indices retain negative-index selection. Zero-dimensional position collections are rejected, but overlay-value broadcasting can accept zero-dimensional arrays as individual values; numerical array outputs retain their array form.

3.3 Zero-Dimensional Arrays

A NumPy scalar such as `np.float64(2)` differs from a zero-dimensional array such as `np.array(2.0)`. Input acceptance depends on the operation. Integer counts, finite-real-scalar checks, and `AstroFrame.pos2xy` position collections reject zero-dimensional arrays. Overlay-value broadcasting extracts the contained NumPy value and repeats it without narrowing its scalar dtype or recursively converting sequence entries.

Coordinate parsing through `coord2xy` accepts a zero-dimensional string array as one coordinate, returning offsets of shape `(2,)`. Conversely, `xy2coord` returns a string for one coordinate pair. Numerical operations such as `Stretcher.do/undo` and noise-density evaluation preserve zero-dimensional array outputs with the computed dtype, including analytic zero limits. Arithmetic can promote a dtype before output normalization.

`AstroData` rejects scalar and zero-dimensional datasets. Trimming and noise estimation reject data that squeeze to a scalar when an image or noise estimate is required; a supplied numeric sigma or `None` bypasses estimation. Generic pass-through helpers do not unwrap arrays or recursively normalize containers. These specialized rules remain local rather than sharing a universal scalar/array converter.

3.4 Finite Real Scalar Inputs

Scalar-only checks for slice length and spacing, scalebar length, resolved `Jy2K` beam and frequency values, fitting sigma, and 3D opacity accept finite Python `int`/`float` and NumPy integer/floating scalars. Booleans, complex values, strings, and arrays (including zero-dimensional arrays) fail these checks with `ValueError`. Validation preserves extended floating precision without converting values. Existing Pydantic coercion of annotated configuration fields remains separate.

The range and optional-value rules remain specific to each argument: `AstroData.slice` requires an explicit positive `length`, rejecting zero and negative lengths; spacing and supplied noise must also be positive; opacity includes zero and one. Gaussian fitting still accepts `sigma=None` for estimation. FITS spectral output accepts `restfreq=None` or zero to retain velocity coordinates, while `Jy2K` requires a positive resolved frequency. Array checks and numerical masks retain their existing roles.

3.5 Basic Array Convention

For a two-dimensional image, the data array is interpreted as

$$I(y, x),$$

where `x` and `y` are one-dimensional arrays giving the spatial coordinates of each pixel. For a three-dimensional spectral cube, the data array is interpreted as

$$I(v, y, x),$$

where `v` is the velocity grid and `x` and `y` are the spatial grids. Thus, the last axis of the NumPy array corresponds to the horizontal coordinate, the second-to-last axis corresponds to the vertical coordinate, and the first axis of a cube corresponds to velocity.

Table 1: Basic data shapes used by `plotastrodata`.

Data type	NumPy shape	Required grids
2D image	`(ny, nx)`	`x`, `y`
3D cube	`(nv, ny, nx)`	`x`, `y`, `v`
PV diagram	`(nv, nx)`	`x`, `v`

When a FITS file is used as input, `plotastrodata` reads the image array and constructs the coordinate grids from the FITS header. In the usual image/cube case, FITS axis 1 is used for `x`, axis 2 for `y`, and axis 3 for `v`. For a position–velocity diagram, set `pv=True`; then axis 1 is treated as the spatial offset axis and axis 2 is treated as the velocity axis.

3.6 Spatial Coordinates

The spatial coordinates `x` and `y` are measured relative to a chosen center. If the input is a FITS file, the center can be read from the FITS WCS information. Alternatively, the user can specify a center manually using a coordinate string such as

```
center = '01h23m45.6s 01d23m45.6s'
```

or with an explicit reference frame such as `J2000`, `B1950`, `FK5`, `FK4`, or `ICRS`.

By default, spatial coordinates are expressed in arcseconds. If the argument `dist` is supplied, the spatial coordinates and beam sizes are multiplied by this value. This is useful, for example, when converting angular offsets in arcseconds to physical offsets in au.

The plotting frame can also be shifted relative to the adopted center using `xoff` and `yoff`. These offsets are applied in the same unit as the spatial grids.

3.7 Velocity Coordinates

For a spectral cube, the velocity grid `v` can be supplied directly or read from the FITS header. When the spectral axis is given in frequency units, `plotastrodata` converts frequency to velocity using the rest frequency. The rest frequency can be given explicitly with `restfreq`, or read from the FITS header. An explicit `restfreq` takes precedence; header lookup prefers `RESTFRQ` over `RESTFREQ`. For `Jy2K`, complete header beam sizes override explicit `bmaj`/`bmin`, with available pixel metadata supplying a fallback beam area.

The systemic velocity `vsys` is subtracted from the velocity grid. Therefore, plotted channel labels usually represent

$$v - v_{\mathrm{sys}}.$$

If no valid rest frequency is available in either the input argument or the FITS header, `plotastrodata` assumes the center of the spectral axis as the rest frequency for frequency-to-velocity conversion.

3.8 FITS Input and Array Input

There are two equivalent ways to give data to the package. The first is to supply a FITS file:

```
from plotastrodata.plot_utils import PlotAstroData as pad

p = pad(fitsimage='cube.fits', rmax=5.0, vmin=-5.0, vmax=5.0)
p.add_color(fitsimage='cube.fits')
```

The second is to supply NumPy arrays directly:

```
p = pad(v=v, rmax=5.0, vmin=-5.0, vmax=5.0)
p.add_color(data=data, x=x, y=y, v=v, beam=[bmaj, bmin, bpa])
```

When arrays are supplied directly, the user should provide the coordinate grids and, when needed, the beam information. The beam is represented as

```
beam = [bmaj, bmin, bpa]
```

where `bmaj` and `bmin` are the positive beam major and minor axes, and `bpa` is the beam position angle in degrees. The beam size should be in the same spatial unit as `x` and `y`. A plotted beam triplet must be complete; use `[None, None, None]` when no beam is available.

Most external uses of `AstroData` involve a single dataset. As a special case, plotting (particularly RGB) can use a list of NumPy arrays with shared coordinate grids and shared metadata or one metadata value per dataset. The processed grids must agree. A nested numeric list is interpreted as one array-like dataset; use a list of NumPy arrays to request multiple datasets. `add_rgb` requires exactly three compatible datasets.

3.9 The `AstroData` Object

Internally, each plotted layer is handled as an `AstroData` object. This object stores the data array, coordinate grids, beam, noise level, brightness unit, and related metadata. Most users do not need to create `AstroData` explicitly for simple plots, because the same information can be passed directly to methods such as `add_color`, `add_contour`, `add_segment`, and `add_rgb`. However, `AstroData` is useful when the user wants to inspect, modify, or reuse the data before plotting.

Table 2: Important `AstroData` quantities.

Quantity	Meaning
`data`	A 2D/3D NumPy array, or a list of arrays for multiple data sets
`x`, `y`	Spatial coordinate grids
`v`	Velocity grid
`fitsimage`	Input FITS file name
`beam`	`[bmaj, bmin, bpa]`, or one such triplet per data set
`sigma`	Noise level, or method used to estimate it
`Tb`	Convert intensity to brightness temperature if `True`
`restfreq`	Rest frequency used for velocity and brightness temperature
`cfactor`	Multiplicative factor applied to the data
`pv`	Treat the data as a position–velocity diagram
`bunit`	Brightness unit of the data

3.10 Swapped Axes and Overlay Angles

`AstroFrame(swapxy=True).read(d)` transposes the last two data axes, exchanges the processed x/y coordinates, and changes the stored beam angle to $90° - \text{bpa}$. It reports the changed order once per read. `AstroData` does not track or undo swaps: later analysis and FITS writing use their usual axis assumptions on the current arrays. In PV mode, `v` and `dv` follow processed y even when it contains positions. The effective PV beam uses the actual velocity spacing before its angle is transformed, with both effective lengths preserved by the swap. `beam_org` retains an independent copy of the original sky beam.

Overlay angles are measured in degrees from the original data $+y$ axis toward $+x$. Apply additional rotation in that frame before `swapxy`; exchanging direction components is equivalent to $90° - \text{angle}$. `xflip` and `yflip` affect only the display, so no extra angle correction is needed. Regions, lines, arrows, and polarization segments remain aligned with the data. The exception is `add_beam`: all its beam angles, including explicitly supplied beams, are already in the processed frame and must not be swapped again.

Sky-coordinate overlay positions require a center; their angular offsets are multiplied by `dist` and exchanged under `swapxy`. Relative numeric positions, including `beampos`, always refer to the displayed axes and are not exchanged.

3.11 Noise and Units

The noise level is stored as `sigma`. It can be supplied as a number, or it can be estimated from the data using one of the supported methods. The noise level is used for contour levels, signal-to-noise based plotting, and some analysis functions.

If `Tb=True`, data in Jy beam^{-1} are converted to brightness temperature using the beam size and rest frequency. This conversion requires sufficient FITS header information or manually supplied metadata. If the necessary beam or frequency information is missing, the conversion may not be possible.

3.12 Position–Velocity Diagrams

A position–velocity diagram is enabled by setting

```
pv = True
```

In this mode, the horizontal axis is the spatial offset and the vertical axis is the velocity. The package treats the second FITS axis as the velocity axis. Some spatial annotations, such as regions, arrows, lines, segments, and scalebars, are not supported in PV mode because the two axes have different physical units.

3.13 Multiple Layers

A single figure can combine multiple layers, such as color maps, contours, segments, and RGB images. These layers may come from different FITS files or arrays. In this case, `plotastrodata` reads each layer as an `AstroData` object and registers it onto the plotting frame. The special value

```
center = 'common'
```

means that the layer should use the common center defined by the plotting frame.

This design makes it possible to overlay data sets with different spatial grids, provided that their coordinates and centers are known.

4 Data Input

The data class AstroData can take a fits file.

```
from plotastrodata.analysis_utils import AstroData

d = AstroData(fitsimage='file_name.fits')
```

It can instead take an array directly, as `AstroData(data=image, x=x, y=y)`, or multiple arrays on the same coordinate grid, as `AstroData(data=[red, green, blue], x=x, y=y)`. In the latter form, each element must be a NumPy array. Arguments such as `Tb`, `sigma`, `beam`, `restfreq`,

`cfactor`, and `bunit` may be shared values or lists with one value per data set; see the "FITS Input and Array Input" subsection for a complete example.

AstroData can take more arguments, such as `Tb` and `sigma`. `Tb=True` means the data values will be converted from flux densities in the unit of Jy beam^{-1} to brightness temperatures in the unit of K. `sigma` specifies how to measure the noise level of the data values: for example, the default option of 'hist' means to use the histogram of the data values. The argument of `pvpa` is the position angle of the PV cut direction in the unit of degree, which will be used to calculate the spatial resolution of the PV diagram.

The data class AstroFrame is necessary to form the AstroData instance in a useful format. AstroFrame can take quantities related to coordinate ranges.

```
from plotastrodata.analysis_utils import AstroFrame

f = AstroFrame(vmin=-5.0, vmax=5.0, vsys=2.0,
               center='00h00m00s 00d00m00s')
```

`vmin`, `vmax`, and `vsys` are in the unit of km s^{-1}. `rmax` is in the unit of arcsec. Instead of `rmax`, other arguments (`xmin`, `xmax`, `ymin`, `ymax`, `xoff`, `yoff`) can be used to adjust the x and y ranges in more detail. When an argument of `fitsimage` is given, the central coordinates `center` is read from the given fits file; the rest frequency is also read from the fits file, which is used for the frequency-velocity conversion. Moreover, an argument of `dist` can specifies the distance used to change the unit of spatial coordinates from arcsec to au. An argument of `swapxy` is used to swap the x and y coordinates. An argument of `pv` must be `True` when the AstroData instance to be read is a position-velocity (PV) diagram, i.e., the first and second axes are the spatial and velocity coordinates. When `quadrants='13'` or `quadrants='24'`, the first and third or second and fourth quadrants of the PV diagram will be averaged, respectively. `xflip` and `yflip` will be used to determine the plotting direction of the x and y axes, respectively; these have a meaning only when a figure is made from the data set.

Forming the AstroData instance needs the following command.

```
f.read(d)
```

After this command, the AstroData instance has useful attributes in the format of numpy array: a 1D array of `d.x` (as well as `d.y` and `d.v`), a 2D or 3D array of `d.data`, a 1D array of `d.beam`, a float of `d.sigma`, a string of `d.bunit`. For multiple inputs, `d.data`, `d.beam`, `d.sigma`, and other per-data-set metadata are lists in the same input order. `d.x` and `d.y` are the relative coordinates in the unit of arcsec from the given `center`. Similarly, `d.v` is the relative coordinate in the unit of km s^{-1} from the given `vsys`. `d.dx`, `d.dy`, and `d.dv` are `d.x[1]-d.x[0]`, `d.y[1]-d.y[0]`, and `d.v[1]-d.v[0]`, respectively. `d.beam` is the beam components `array([bmaj, bmin, bpa])`, where `bmaj` and `bmin` are in the unit of arcsec, while `bpa` is in the unit of degree. When `Tb=True` above, the data values are converted to the brightness temperature and stored as `d.data`. `d.sigma` is the noise level measured in the way specified above in the same unit as `d.data`. `d.bunit` is the unit of `d.data`. AstroData can take two more arguments, `restfreq` and `cfactor`. `restfreq` is used to explicitly set the rest frequency in the unit of Hz, which is used to convert frequency to velocity and Jy beam^{-1} to K. When the unit of the frequency/velocity axis in the input fits header is m/s, `f.read(d)` divides the coordinate by 1000. The argument `cfactor` specifies a constant factor that `d.data` is multiplied by. This will be useful when one wants to change the intensity unit from Jy beam^{-1} to mJy beam^{-1}.

In addition to the input attributes, `f.read(d)` adds three attributes to the AstroData instance `d`: `fitsimage_org`, `sigma_org`, and `fitsheader`. `fitsimage_org` saves the input `fitsimage`, while `fitsimage` is updated to None after `f.read(d)`. Similarly, `sigma_org` saves the input `sigma`, which may be a string, while `sigma` is updated to the calculated value. `fitsheader` is the fits header in the format of astropy.io.fits.open(fitsimage)[0].

When `pv=True` in `AstroFrame`, the `read` method changes the `beam` attribute of the AstroData instance. The first element will be the velocity resolution. The second element of `beam` will be $1/\sqrt{\cos^2(\mathrm{bpa}-\mathrm{pvpa})/\mathrm{bmaj}^2+\sin^2(\mathrm{bpa}-\mathrm{pvpa})/\mathrm{bmin}^2}$. This is calculated from the intersection of the beam ellipse and the PV cut line. The third element will be 0; thus, the velocity resolution (first element) is regarded as the vertical length by default when this "beam" is plotted in a PV diagram. The original beam will be stored in `beam_org`.

4.1 Noise measurement

The process of reading data includes a step of noise measurement, unless `sigma` is given as a specific value. AstroData can take the following methods for noise measurement, e.g., `sigma='edge,neg,hist-pbcor'`.

Multiple options of the following four can be used to specify how to select the values used for the noise estimation.

edge: This option means that the first and last channels are used.

out: This option means that the first 20% and the last 20% pixels in the x and y directions are used. In other words, the central square of 60% × 60% is removed.

neg: This option means that pixels with negative values are used.

iter: This option first calculates the mean and standard deviation, and then picks up pixels with values within ±3.5× "standard deviation" from "mean". The new mean and standard deviation are calculated only with the picked up pixels. This step is iterated five times to suppress the effect of the signal.

One of the following three can be used to specify how to estimate the noise. If none of them is selected, simply the mean and standard deviation are calculated.

med: This option calculates the standard deviation from the median of the squared values by assuming Gaussian noise.

hist: This option fits the histogram (probability density) with a normalized Gaussian function to obtain the mean and standard deviation.

$$P(x)=\frac{1}{\sqrt{2\pi}\sigma}\exp\left(-\frac{1}{2}\left(\frac{x-\mu}{\sigma}\right)^2\right) \tag{1}$$

hist-pbcor This option is applicable to a dataset after primary beam correction. To obtain the analytical function of the model, the model assumes that the pixels are within a circle with a radius of r_{out}. The primary beam has half width at half maximum r_p. The primary beam correction is expressed as 2^{R^2}, where $R:=r/r_p$. The fitted parameter named `R` in `gauss_pbcor` is the dimensionless outer radius $R_{\mathrm{out}}=r_{\mathrm{out}}/r_p$, rather than the radius scaled by the full width at half maximum. The primary-beam response is $B(r)=2^{-(r/r_p)^2}$, so $B(r_p)=1/2$. The probability

function is calculated as follows:

$$P(x) = \frac{1}{\pi R_{\rm out}^2} \int_0^{R_{\rm out}} \frac{1}{\sqrt{2\pi}\sigma 2^{R^2}} \exp\left[-\frac{1}{2}\left(\frac{x-\mu 2^{R^2}}{\sigma 2^{R^2}}\right)^2\right] 2\pi R dR \tag{2}$$

$$= \frac{1}{R_{\rm out}^2} \int_0^{R_{\rm out}^2} \frac{2^{-S}}{\sqrt{2\pi}\sigma} \exp\left[-\frac{1}{2}\left(\frac{x2^{-S}-\mu}{\sigma}\right)^2\right] dS \tag{3}$$

$$= \frac{1}{R_{\rm out}^2 x \ln 2} \int_{x2^{-R_{\rm out}^2}}^{x} \frac{1}{\sqrt{2\pi}\sigma} \exp\left[-\frac{1}{2}\left(\frac{T-\mu}{\sigma}\right)^2\right] dT \tag{4}$$

$$= \frac{1}{xR_{\rm out}^2 2\ln 2} \int_{(x2^{-R_{\rm out}^2}-\mu)/\sqrt{2}\sigma}^{(x-\mu)/\sqrt{2}\sigma} \frac{2}{\sqrt{\pi}} e^{-T^2} dT \tag{5}$$

$$= \frac{\mathrm{erf}\left(\frac{x-\mu}{\sqrt{2}\sigma}\right) - \mathrm{erf}\left(\frac{x2^{-R_{\rm out}^2}-\mu}{\sqrt{2}\sigma}\right)}{xR_{\rm out}^2 2\ln 2} \tag{6}$$

This function satisfies the normalization.

$$\int_{-\infty}^{\infty} P(x)dx = \frac{1}{R_{\rm out}^2 2\ln 2} \int_{-\infty}^{\infty} \frac{1}{x} \int_{\frac{x2^{-R_{\rm out}^2}-\mu}{\sqrt{2}\sigma}}^{\frac{x-\mu}{\sqrt{2}\sigma}} \frac{2}{\sqrt{\pi}} e^{-T^2} dT dx \tag{7}$$

$$= \frac{1}{R_{\rm out}^2 2\ln 2} \int_{-\infty}^{\infty} \int_{\sqrt{2}\sigma(T+\mu)}^{\sqrt{2}\sigma(T+\mu)2^{R_{\rm out}^2}} \frac{1}{x}\frac{2}{\sqrt{\pi}} e^{-T^2} dx dT \tag{8}$$

$$= \frac{1}{R_{\rm out}^2 2\ln 2} \int_{-\infty}^{\infty} \frac{2}{\sqrt{\pi}} e^{-T^2} \int_{\sqrt{2}\sigma(T+\mu)}^{\sqrt{2}\sigma(T+\mu)2^{R_{\rm out}^2}} \frac{1}{x} dx dT \tag{9}$$

$$= \frac{1}{R_{\rm out}^2 2\ln 2} \int_{-\infty}^{\infty} \frac{2}{\sqrt{\pi}} e^{-T^2} \ln\left(2^{R_{\rm out}^2}\right) dT \tag{10}$$

$$= \frac{1}{R_{\rm out}^2 2\ln 2} \int_{-\infty}^{\infty} \frac{2}{\sqrt{\pi}} e^{-T^2} R_{\rm out}^2 \ln 2 \; dT \tag{11}$$

$$= 1 \tag{12}$$

The fitting to the histogram minimizes the following χ^2.

$$\chi^2 = \sum_i \frac{(O_i - M_i)^2}{\max(M_i, 1)}, \tag{13}$$

where O_i and M_i are the number of points in the i-th bin of the observed and model histograms, respectively.

Here the expected count is $M_i = N\Delta x\, p_i$, where N is the number of samples inside the histogram edges and p_i is the model density normalized over those edges. Counts set the statistical weight and do not cancel from χ^2.

`Noise.gen_histogram` and `fit_histogram` support unweighted histograms with equally spaced bins, with either `density=True` (the default) or `density=False`. Fitting uses raw counts in

both cases. Weights and uneven edges are rejected; widths are compared with their mean using `rtol=1e-5, atol=0`, allowing float32 rounding. Histogram validation distinguishes malformed edge arrays, nonfinite or nonpositive bin widths, and finite positive but unequal widths, with separate `ValueError` messages. Even finite edges can overflow during subtraction, so width validation is also necessary. The `edges` argument of `noise_utils.normalize` is keyword-only. These restrictions apply to noise fitting, not the general `AstroData.histogram` wrapper.

The noise estimation can be performed with adjusted parameters by using the class of plotastrodata.noise_utils.Noise.

```
from plotastrodata.noise_utils import Noise

x = np.linspace(-1.5, 1.5, 301)
y = np.linspace(-1.5, 1.5, 301)
x, y = np.meshgrid(x, y)
r = np.hypot(x, y)
data = np.random.randn(*np.shape(r))
data[r > 1.5] = np.nan
data = data * np.exp2(r**2)
n = Noise(data=data, sigma='hist-pbcor')
n.gen_histogram()
n.fit_histogram()
print(n.std)
# 0.989151253878513
```

Figure 1 shows the histogram of the noise in the above example.

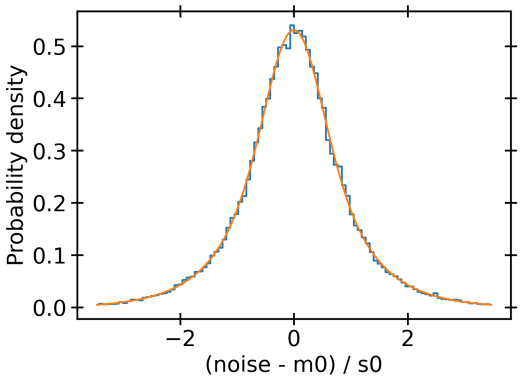


Figure 1: Example of noise histogram and model function with primary beam correction.

5 Analysis Methods

AstroData also has handy methods to analyze the 2D/3D data.

The `binning` method rebins the 2D/3D data with a given width for each coordinate.

```
d.binning(width=[5, 4, 2])
```

This command takes an average over 5 channels in the velocity direction, 4 pixels in the y direction, and 2 pixels in the x direction. The number of channels and pixels are decreased accordingly after this method. If the length of `width` is 2, the two values are regarded as the widths for the y and x directions. Binning in the v-axis updates the noise level (`d.sigma`) by dividing it by the square root of the number of binned channels. Binning in the x- or y-axis does not update the noise level.

The `centering` method sets the coordinates so that the spatial center and the systemic velocity have the exact zero coordinates by interpolation.

```
d.centering(includexy=True, includev=False)
```

This command adjusts the x and y coordinates but does not adjust the velocity coordinate. This method will be useful when one wants to quickly get a radial profile along a line passing the center or a PV diagram (using the `rotate` method together).

The `circularbeam` method makes the beam shape circular by additional 2D Gaussian convolution. This method takes no argument.

```
d.circularbeam()
```

The new beam has the major and minor axes same as the old major axis. This method also updates the attribute of `d.beam` accordingly.

The `deproject` method deprojects the 2D/3D data with a given position angle (P.A.) and inclination angle.

```
d.deproject(pa=45, incl=45)
```

`pa` is the position angle from the north to the east in the unit of degree. `incl` is the inclination angle; `incl=0` means the face-on configuration and thus no deprojection. This command replaces `d.data` and `d.beam` with the deprojected data and the deprojected beam, respectively.

The `fit2d` method performs MCMC fitting to the 2D image of `d.data` or `d.data[chan]`, where `chan` is the channel number, by using emcee or ptemcee through the class of `plotastrodata.fitting_utils.EmceeCorner`.

```
res = d.fit2d(chan=12, model=model, bounds=bounds,
              kwargs_fit={'nwalkersperdim': 2, 'nsteps': 1000, 'nburnin': 500},
              kwargs_plotcorner={'savefig': 'corner.png'})
```

`model` is a function with the shape of `model(par, x, y)`, where `par` is the list of parameters. `bounds` is the 2D list of the parameter boundary in the shape of $[[p_{0,\min}, p_{0,\max}], [p_{1,\min}, p_{1,\max}], ...]$. `kwargs_fit` and `kwargs_plotcorner` are the arguments for the methods of `EmceeCorner.fit` and `EmceeCorner.plotcorner`, respectively. `nsteps` includes the number the burn-in steps. The output `res` is a dictionary consisting of `popt`, `plow`, `pmid`, `phigh`, `model`, and `residual`. The first four keys provide the best, lower percentile, 50 percentile, and higher percentile parameters. The last two keys provide the model and residual 2D images.

The `histogram` method returns the bins and the histogram in the bins of the attribute of `d.data` by using numpy.histogram().

```
hbin, hist = d.histogram(bins=10)
```

The arguments are the same as those for numpy.histogram(). The bins (`hbin`) have the same length as the histogram (`hist`), which is different from the original numpy.histogram.

The gaussfit2d method performs the 2D Gaussian fitting to the 2D image of d.data or d.data[chan], where chan is the channel number, by using fitting_utils.gaussfit2d.

```
res = d.gaussfit2d(chan=12)
```

This command performs the fitting to the channel of 12 in d.data. For 2D data, the chan argument can be omitted. The output (res here) is a dictionary having keys of popt, perr, model, residual, and center. popt and perr are the optimized parameters (peak intensity, central x, central y, major FWHM, minor FWHM, and P.A.) and their errors. The model and residual are 2D arrays with the same shape as the fitted 2D array. center is the best-fit center in the format of "hms dms" string. This central coordinates are calculated from d.center and the best-fit central x and y through coord_utils.xy2coord.

The mask method puts numpy.nan on pixels that satisfies a given condition.

```
d.mask(dataformask=d2.data, includepix=[1e-3, 100], excludepix=[50, 200])
```

This command puts numpy.nan on pixels of d.data where d2.data is outside [1e-3, 100] or inside [50, 200], where d2 is an AstroData instance different from d. This method will be useful when one wants to put a mask on a moment 1 map using the values of a moment 0 map.

The moment method stores moment maps from a real 3D cube in the current (v, y, x) axis order, including after a swapped read.

```
d.moment(vmask=None, threshold_cube=3, threshold_mom0=3)
# Results: d.mom0, d.mom1, d.mom2, d.sigma_mom0
```

The optional vmask is a one-dimensional NumPy Boolean array matching d.v, with at least one selected channel; None selects all. A finite velocity grid, finite positive scalar noise sigma, and finite nonzero scalar dv are required. The grid must be monotonic and uniformly spaced with spacing magnitude |dv| (relative tolerance 10^{-5}); a single channel only requires a valid dv. NaNs in the data are ignored; infinities are rejected. Both thresholds must be finite nonnegative real scalars, excluding booleans and arrays. The (y, x) array mom0 is the sum of selected intensities times |dv|, without intensity thresholding. Its scalar noise sigma_mom0 is $\sqrt{N}$ sigma |dv|, assuming independent noise in the N selected channels, even where data are NaN. This scalar is a Python float, except that extended-precision NumPy floats retain their precision. mom1 and mom2 are the intensity-weighted mean velocity and standard deviation (not FWHM), using intensities at or above threshold_cube * sigma. They are NaN where mom0 is below threshold_mom0 * sigma_mom0 or no positive weight survives. All-NaN spectra have zero mom0. The method returns None and does not modify the input arrays.

The profile method makes line profiles at given spatial positions.

```
v, f, g = d.profile(xlist=[-0.5, 0.2], ylist=[1, 0.8],
                    ellipse=[[0.3, 0.2, 45], [0.3, 0.2, 45]])
```

This command makes line profiles at (x,y)=(-0.5, 1) and (0.2, 0.8) in the unit of arcsec. The intensity of each profile is an average over a boxcar ellipse with the major axis of 0.3 arcsec, the minor axis of 0.2 arcsec, and the P.A. of 45 degrees. Instead of xlist and ylist, coordinate strings can be input using an argument of coords: coords=['00h00m00.0s 00d00m00.0s', '11h11m11.1s 11d11m11.1s']. When ellipse is omitted, the profile is made by picking up the values at the closest pixel to the given position. When the ellipse size is not so large compared to the pixel size, the integer argument ninterp may be useful; this makes the pixel size ninterp times finer

by interpolartion. When the boolian argument `flux` is True, the output profile has the unit of Jy. Additionally, when the boolian argument `gaussfit` is True, the profiles will be fitted with a 1D Gaussian function. The output `v` above is `d.v`. The output `f` above is a list of 1D-array profiles, i.e., 2D array. The output `g` is a list of dictionaries; each dictionary has keys of `best` and `error` (square root of the diagonal components of the covarience).

The `rotate` method rotates the attribute `d.data` by the given angle in the unit of degree by interpolation.

```
d.rotate(pa=45)
```

When a pixel refers to a position outside the original image, this pixel has numpy.nan after this method.

The `slice` method requires an explicit finite positive `length` (no default). It makes a radial profile for a 2D image or a PV diagram for a 3D image along a given direction and length by interpolation.

```
r, f = d.slice(length=3, pa=45, dx=0.2)
```

This command makes a radial profile (or PV diagram) along a cut with a length of 3 arcsec at P.A.=45 degrees with a separation of 0.2 arcsec. The output `r` is a 1D array of the positional offsets. For a 2D image, `f` is a 1D array of the intensity at those positions; for a 3D cube, `f` is a 2D array with the shape of `(len(d.v), len(r))`. When the separation `dx` is omitted, the absolute value of the x pixel size is adopted.

The `todict` method returns the attributes as a dictionary. Contained lists and NumPy arrays are shared with the object, not copied.

```
a = d.todict()
```

This output includes keys of `data`, `x`, `y`, `v`, `fitsimage`, `beam`, `Tb`, `restfreq`, `cfactor`, `sigma`, `center`, `pv`, and `bunit`. `pv` means whether the data array is a position-velocity diagram. This dictionary can be input to methods of PlotAstroData as `**a`.

The `writetofits` method exports the instance as a fits file.

```
d.writetofits(fitsimage='new_file_name.fits')
```

The output fits file reuses the header components of the fits file used to make the AstroData instance (`'file_name.fits'` above); some header components are automatically updated after the above-mentioned methods for analysis, such as `CDELT1`.

5.1 FITS Conversion and Cached Data

`FitsData.get_data` returns cached data; its keyword arguments are applied only when data have not yet been generated. Call `gen_data` explicitly to change options:

```
from plotastrodata.fits_utils import FitsData
fd = FitsData('image.fits')
fd.gen_data(Tb=True)
temperature = fd.get_data()
fd.gen_data(Tb=False)
original_units = fd.get_data()
```

Temperature and logarithmic conversion leave the source HDU array unchanged. Repeated temperature conversion starts from the original data; integer inputs are promoted to floating point as needed.

`AstroData.writetofits` converts spatial offsets from the current distance-scaled units back to degrees even when `center=None`. It uses the current axis order and does not undo `swapxy`.

6 Plotting Methods

The class PlotAstroData can take an AstroData instance through the method of `d.todict()`.

```
from plotastrodata.plot_utils import PlotAstroData

p = PlotAstroData(rmax=3.0)
p.add_color(**d.todict())
p.add_scalebar(length=50 / 140, label='50 au')
p.set_axis()
p.savefig('figure_name.png')
```

These commands make a color map using the AstroData instance `d`. PlotAstroData can take the same arguments as AstroFrame to define the plotting ranges; particularly `rmax` (also, `vmin` and `vmax` for a cube or a PV diagram) is necessary. The method `p.add_color()` can take a fits file directly instead of the AstroData instance, as `p.add_color(fitsimage='file_name.fits')`. This method can actually take the same arguments as AstroData to specify the data to be plotted as a color map; `**d.todict()` does this indirectly. The command `p.add_scalebar()` can be omitted if the map does not need to show a scale bar. The command `p.set_axis()` (or `p.set_axis_radec()`) is necessary even without any argument.

The class PlotAstroData can take more arguments. `veldigit` specifies the number of digits of the velocity label on the channel maps. `channelnumber` specifies which channel is used to make a 2D image; when this is None (default), channel maps are made instead of a 2D image. `nrows` and `ncols` specify the grid shape of the channel maps. `fontsize` specifies the basic font size in the figure. `nancolor` specifies which color is used when the value is nan. `dpi` specifies the resolution of a raster image file. `figsize` is the same as the argument for Figure.figure of matplotlib. `fig` and `ax` must be supplied together or both omitted. External Figure/Axes objects support only one panel on one page. A selected `channelnumber` retains plotting-data shape `(1, ny, nx)`, although the displayed image is two-dimensional. More detailed usage can be found in the `example.py` file and `https://plotastrodata.readthedocs.io/en/latest/#`. The following is the explanation of each method of PlotAstroData.

6.1 Core Class Parameters

Tables 3-5 summarize the main parameters used by the core classes. The most common workflow is to define the plotting frame with `PlotAstroData`, and then pass data-specific parameters to methods such as `add_color`, `add_contour`, `add_segment`, and `add_rgb`. Internally, these data-specific parameters are handled through `AstroData`.

Table 3: Main parameters of `AstroData`.

Parameter	Default	Description
`data`	`None`	One 2D/3D NumPy array, an array-like single data set, or a list of NumPy arrays for multiple data sets.
`x`	`None`	One-dimensional horizontal spatial grid.
`y`	`None`	One-dimensional vertical spatial grid.
`v`	`None`	One-dimensional velocity grid.

Parameter	Default	Description
beam	(None, None, None)	Beam information as a complete [bmaj, bmin, bpa] triplet, the all-None no-beam triplet, or a list of triplets corresponding to a list of data sets.
fitsimage	None	Input FITS file name. If given, the data and grids are read from the FITS file.
Tb	False	If True, convert Jy beam^{-1} data to brightness temperature. A scalar is shared; a list applies values per data set.
sigma	'hist'	Noise level, or method used to estimate the noise. A scalar is shared; a list applies values per data set.
center	'common'	Coordinate center. 'common' uses the center of the plotting frame.
restfreq	None	Rest frequency used for velocity conversion and brightness temperature conversion.
cfactor	1	Multiplicative factor applied to the data and noise.
pvpa	None	Position angle of the cut used for a PV diagram.
pv	False	If True, treat the data as a position–velocity diagram.
bunit	''	Brightness unit of the data. A string is shared; a list applies units per data set.

Table 4: Main parameters of AstroFrame.

Parameter	Default	Description
rmax	1e10	Half-size of the plotting region. Used to set both x and y ranges unless explicit limits are given.
xmax, xmin	None	Explicit horizontal plotting limits.
ymax, ymin	None	Explicit vertical plotting limits.
dist	1	Multiplicative factor for spatial coordinates and beam sizes. Useful for converting arcsec to au.
center	None	Coordinate center of the plotting frame.
fitsimage	None	FITS file used to read the center when center is not supplied.
xoff	0	Horizontal offset of the plotting region from the center.
yoff	0	Vertical offset of the plotting region from the center.
vsys	0	Systemic velocity subtracted from the velocity grid.
vmin	-1e20	Minimum velocity included in the plotting frame.
vmax	1e20	Maximum velocity included in the plotting frame.
xflip	True	If True, the x-axis is plotted with positive values to the left, as commonly used for R.A. offsets.
yflip	False	If True, reverse the y-axis direction.
swapxy	False	If True, swap the x and y axes.
pv	False	If True, use position–velocity plotting mode.

Parameter	Default	Description
quadrants	None	Set to '13' or '24' to average symmetric quadrants; no other value is accepted.

6.2 Validated Configuration

plotastrodata uses Pydantic (version 2 or later) to validate selected public plotting configurations when they are created. This dependency is installed with the package. Invalid values therefore raise an error before data are plotted. The validated options are the stretch type, x- and y-axis scales, a positive loglog range, quadrants, and the beam display arguments described below. NumPy arrays supplied as beam triplets are converted to the equivalent Python-list representation before validation.

Table 5: Main parameters of PlotAstroData.

Parameter	Default	Description
v	None	Velocity grid used to set up channel maps when it is not read from a FITS file.
vskip	1	Use every vskip-th velocity channel.
veldigit	2	Number of digits after the decimal point in channel labels.
restfreq	None	Rest frequency used when reading velocity information from FITS data.
channelnumber	None	If an integer is given, plot only that channel. This is also useful for animation.
nrows	4	Number of rows in a channel-map page.
ncols	6	Number of columns in a channel-map page.
fontsize	None	Matplotlib font size. If None, a default value is chosen depending on the plot type.
nancolor	'w'	Background color used for masked or NaN regions.
dpi	256	Output resolution for saved figures.
figsize	None	Figure size. If None, a suitable size is estimated from the plotting range.
fig	None	External Matplotlib figure object.
ax	None	External Matplotlib axis object. Mainly used for custom 2D plotting layouts.
**kwargs	–	Additional plotting-frame parameters passed to AstroFrame, such as rmax, center, vmin, vmax, xflip, and pv.

6.3 Arguments for the methods to make maps

The four methods to make maps (add_color(), add_contour(), add_segment(), and add_rgb()) share many arguments. This subsection overviews those shared arguments.

```
data:      2D or 3D numpy array or None.
x:         1D numpy array or None.
```

```
y:          1D numpy array or None.
v:          1D numpy array or None.
beam:       1D numpy array ([bmaj, bmin, bpa]) or [None, None, None].
fitsimage:  File name or None. If this is not None, this will overwrite data, x, y,
            v, and beam.
Tb:         Bool. Whether to convert the data from an intensity (Jy/beam) to
            a brightness temperature (K).
sigma:      Method name or a noise value or None. See also Section 1.1 Noise
            measurement.
center:     Central coordinates or 'common'. 'common' means to copy the coordinates
            from AstroFrame in AstroFrame.read(AstroData).
restfreq:   Rest frequency in Hz. If this is not None, this value is given priority
            over the one in the fits file.
cfactor:    Float factor to be multipled by the data array.
pvpa:       Position angle of the cutting direction when the data is
            a position-velocity diagram.
(These are the aguments for the AstroData class.)
```

```
beam:        [bmaj, bmin, bpa], a NumPy array containing one triple, or a
             list of such triples. A triple is complete with bmaj > 0 and
             bmin > 0, or is [None, None, None] when no beam is available.
             Partial triples are not accepted.
show_beam:   Bool. Whether to show the beam.
beamcolor:   Color name for the beam, or a list with one color per beam.
             One color is shared by all beams.
beampos:     Relative [x, y] position, a list with one position per beam,
             or None. One position is shared by all beams. Each coordinate
             must be from 0 (left or bottom) to 1 (right or top).
beam_kwargs: Additional arguments for matplotlib.patches as a dictionary.
             For example, facecolor, edgecolor, linewidth, alpha, hatch, etc.
(These are the arguments for the Beam class.)
```

```
stretch:      Stretch type: linear, log, asinh, or power. See also Figure 2.
stretchscale: Scaling float factor, or one factor per RGB channel, for asinh.
vmin:         Minimum value, or one value per RGB channel, or None.
vmax:         Maximum value, or one value per RGB channel, or None.
sigma:        Float noise level, a list of noise levels, or None. This will be
              used as vmin when vmin is not given for the log or power stretch.
(These are the arguments for the Stretcher class.)
```

```
xskip: Integer to specify how many pixels are skipped in the x-direction.
yskip: Integer to specify how many pixels are skipped in the y-direction.
```

The AstroData arguments are always necessary for the four mapping methods. The stretch arguments can be used only for `add_color()` and `add_rgb()`. For `add_rgb()`, the stretch and beam arguments may be a list of three elements.

6.4 Maps

The `add_color` method plots the given data in the format of a color map.

```
p.add_color(**d.todict(), stretch='log', show_cbar=True, cblabel=r'mJy beam$^{-1}$',
            cbticks=[0.01, 0.03, 0.1, 0.3], cbticklabels=['10', '30', '100', '300'],
            cblocation='right')
```

The `stretch` argument can be 'linear', 'log', 'asinh', or 'power'. The Arcsin hyperbolic (asinh) stretch can be adjusted by another argument of `stretchscale` as asinh(data / stretchscale). The power-law stretch can be adjusted by another argument of `stretchpower` as dataˆstretchpower / stretchpower. The differece by `stretch` is shown in Figure 2. Lower (higher) indices for the power-law stretch zoom into weaker (stronger) signals. The asinh stretch can show weak signals in the linear scale and strong signals in the logarithmic scale simultaneously (the noise level in Figure 2 is 0.01). The arguments starting with 'cb' adjust the colorbar. In addition, two arguments, `xskip` and `yskip`, can be used to skip spatial pixels in this method as well as `add_contour`, `add_segment`, and `add_rgb`. Similarly, `show_beam`, `beamcolor`, and `beampos` can be used in these four methods to adjust beam color and position, as well as switch whether to show the beam.

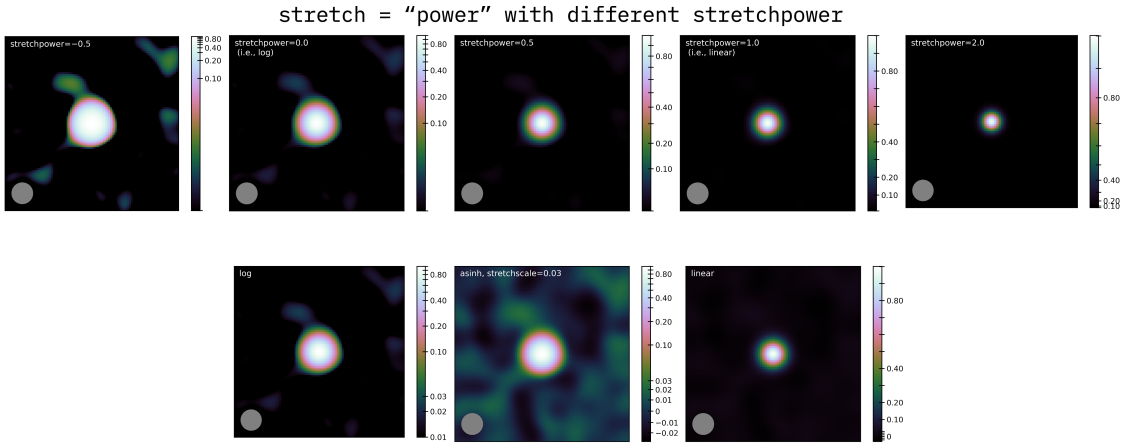


Figure 2: Difference in the color map due to `stretch`.

The `add_contour` method plots the given data in the format of a contour map.

```
p.add_contour(**d.todict(), levels=[-3, 3, 6, 9])
```

The `levels` argument spedifies the contour levels in the unit of `d.sigma`.

The `add_rgb` method plots the given data in the format of three-color maps. The input three data sets are mixed as Red, Green, and Blue.

```
p.add_rgb(**d.todict(), stretch=['linear', 'log', 'linear'])
```

For this method, the input is a combination of three data sets, and thus `d.data` here must be a 1D list (not a numpy array) of three 2D/3D numpy arrays. In the same way as `add_color`, `stretchscale` and `stretchpower` are available in this method.

The `add_segment` method plots the given data in the format of a segment map, as is often used for polarization maps.

```
p.add_segment(Qfits='Qfile_name.fits', Ufits='Ufits_name.fits',
              ampfactor=3.8, angonly=False, rotation=90, cutoff=3.0)
```

The input format of the data is different from that for `add_color` and `add_contour`. One of the following pairs must be given: (`ampfits`, `angfits`), (`Qfits`, `Ufits`), (`amp`, `ang`), or (`stQ`, `stU`). The

latter two pairs are supposed to be in the numpy.array format. The `ampfactor` argument can be used to adjust the segment length. When `angonly=True`, the segment length is set to be uniform. The `rotation` can be used to rotate the segments in the unit of degree. The `cutoff` argument specifies the intensity threshold to calculate the amplitude and angle from the Stokes Q and U intensities.

With default quiver settings, the maximum segment length is `ampfactor * abs(dx) / scale`, where `dx` is the displayed horizontal pixel spacing after sampling and swapping. `scale` is a finite positive dimensionless inverse-length factor, defaulting to 1. Thus unit amplitude normalization and `scale=1` give a one-pixel maximum length; `scale=2` halves it.

6.5 Patches, markers, text, lines, arrows, and scale bar

The class PlotAstroData also has methods for plotting regions (patches), markers, text, lines, and arrows. The following is the explanation of each method for this purpose.

The `add_region` method overlays an ellipse or a rectangle. For both shapes, the supplied position is the geometric center.

```
p.add_region(patch='ellipse',
             poslist=[[0.5, 0.5], [0.1, 0.2]],
             majlist=[0.3, 1.0],
             minlist=[0.2, 0.8],
             palist=[0, 90])
```

The argument `patch` may be 'ellipse' or 'rectangle'. The patch position can be specified by a 2D list like [0.1, 0.2] or a coordinate string like '01h23m45.6s 01d23m45.6s'. The list means a fractional position from left to right and bottom to top: e.g., [0, 0], [0.5, 0.5], and [1, 1] are bottom left, center, and top right, respectively. `majlist`, `minlist`, and `palist` specify the patch size in current spatial units (arcsec or au) and the orientation in degrees. Other arguments for matplotlib.patches.Ellipse or matplotlib.patches.Rectangle can be used in this method. Another method `add_beam` uses this method and is executed internally in `add_color`, `add_contour`, `add_segment`, `add_rgb`.

The `add_marker` method overlays markers.

```
p.add_marker(poslist=[[0.5, 0.5], [0.1, 0.2]])
```

Except for the position list `poslist` (same as in `add_region`), the marker properties can be specified by the arguments for Axes.plot of matplotlib.

The `add_text` method overlays text.

```
p.add_text(poslist=[[0.5, 0.5], [0.1, 0.2]],
           slist=['abs', 'hoge'])
```

Except for the position list `poslist` (same as in `add_region`) and the list of text `slist`, the text properties can be specified by the arguments for Axes.text of matplotlib.

The `add_line` method overlays lines.

```
p.add_line(poslist=[[0.5, 0.5], [0.1, 0.2]],
           anglelist=[0, 90],
           rlist=[0.4, 1])
```

The position list `poslist` (same as in `add_region`) specifies the start position of the line. `anglelist` and `rlist` specify the position angle in the unit of degree and the length in current spatial units

(arcsec or au) of the line, respectively. Other arguments for Axes.plot of matplotlib can be used in this method.

The `add_arrow` method overlays arrows.

```
p.add_arrow(poslist=[[0.5, 0.5], [0.1, 0.2]],
            anglelist=[0, 90],
            rlist=[0.4, 1])
```

`poslist`, `anglelist`, and `rlist` are the same in `add_line`. Other arguments for Axes.quiver of matplotlib can be used in this method.

The `add_scalebar` method overlays a scale bar.

```
p.add_scalebar(length=0.5,
               label='70 au',
               barpos=[0.8, 0.1])
```

The bar length `length` is required and must be finite and positive in horizontal data-axis units (arcsec, or au after distance scaling). PV diagrams are rejected. The bar label `label` is located above the bar. `barpos` is a fractional position from left to right and from bottom to top (same as `poslist`). In addition, the arguments of `color`, `fontsize`, `linewidth`, and `bbox` can be used in this method. `bbox` is a dictionary argument for Axes.text of matplotlib.

6.6 Setting axes and saving figures

The method `set_axis` (or `set_axis_radec`) is necessary even without any argument. This method provides axes of relative coordinates and can also adjust things related to the axes.

```
p.set_axis(title='figure title')
```

The `title` argument can be either string or dictionary; the latter is a set of arguments for Axes.set_title or Figure.suptitle of matplotlib. This method can also take more arguments related to Axes.set_* of matplotlib: `xscale`, `yscale`, `xlim`, `ylim`, `xlabel`, `ylabel`, `xticks`, `yticks`, `xticklabels`, and `yticklabels` use the same input format as in Axes.set_* of matplotlib. The supported scale values are `'linear'`, `'log'`, `'symlog'`, `'asinh'`, and `'logit'`. When `xscale='log'` or `yscale='log'`, `plotastrodata` labels decade ticks and major ticks near the plotted limits; intermediate logarithmic ticks are shown as unlabeled minor ticks. Explicit `xticks` or `yticks` override this automatic major-tick selection. `xticksminor` and `yticksminor` accept explicit position lists or positive Python/NumPy integer subdivision counts per major interval. Counts reject booleans, floats, and arrays. Generating subdivisions requires at least two major ticks; the count is validated when the axis settings are constructed. Two more dictionary arguments, `grid` and `aspect`, are available as for Axes.grid and Axes.set_aspect of matplotlib.

The convenience argument `loglog` accepts a positive value, applies logarithmic scaling to both axes, and uses the same tick-labeling behavior. Its value gives the dynamic range of each axis: the lower limit is set to the upper limit divided by `loglog`. For example,

```
p.set_axis(title='log-log PV diagram', loglog=20)
```

plots both axes logarithmically over a factor of 20 and uses the same tick positions on the x and y axes.

The method `set_axis_radec` provides right ascension and declination coordinates. It requires a known center, `dist=1`, and non-PV data. With `swapxy=True`, Dec. labels move to x and R.A. labels to y. The center frame is preserved.

```
p.set_axis_radec(title='figure title',
                 xlabel='R.A. (ICRS)',
                 ylabel='Dec. (ICRS)',
                 nticksminor=2,
                 grid=None)
```

`title`, `xlabel`, `ylabel`, and `grid` are the same as in `set_axis`. `nticksminor` specifies how many times more ticks are needed than the major ticks.

The method `savefig` saves the figure as an image file and/or shows it on the screen.

```
p.savefig(filename='hoge.png',
          show=True)
```

In addition to the two arguments, the arguments for Figure.savefig of matplotlib can be used for this method. After optional saving and display, this method closes only the figures managed by the `PlotAstroData` instance; unrelated Matplotlib figures remain open.

The method `get_figax` returns the Figure and Axes objects for a single-page plot. It is not supported for multi-page channel maps. Call this method before `savefig` when the Figure or Axes needs further modification, because `savefig` closes the figures managed by the instance.

```
fig, ax = p.get_figax()
```

Figures 3 and 4 show examples obtained through the `plotastrodata.plot_utils` module. They might appear to be messy, but this is because these examples aim to show various features of this module. The module enables you to fine-tune the figures in various aspects.

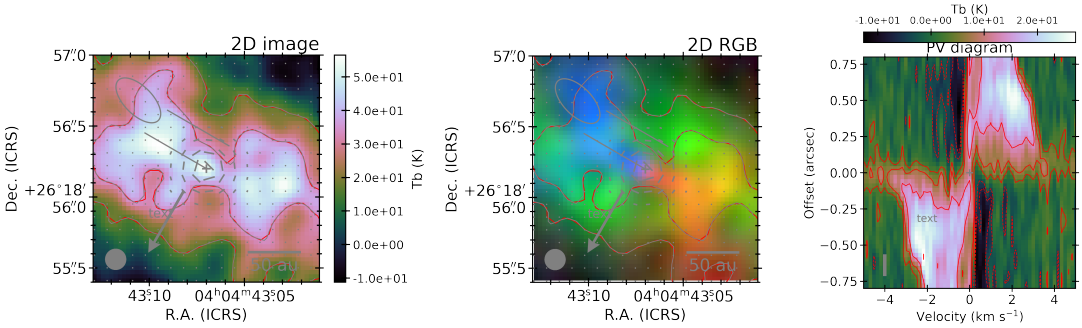


Figure 3: Examples of 2D images. In addition to the color and RGB maps, scale bars, and beams, the left and middle panels show contour maps, segment maps, regions, markers, lines, arrows, and text. The right panel is a position-velocity diagram (color and contour maps) with a marker and text.

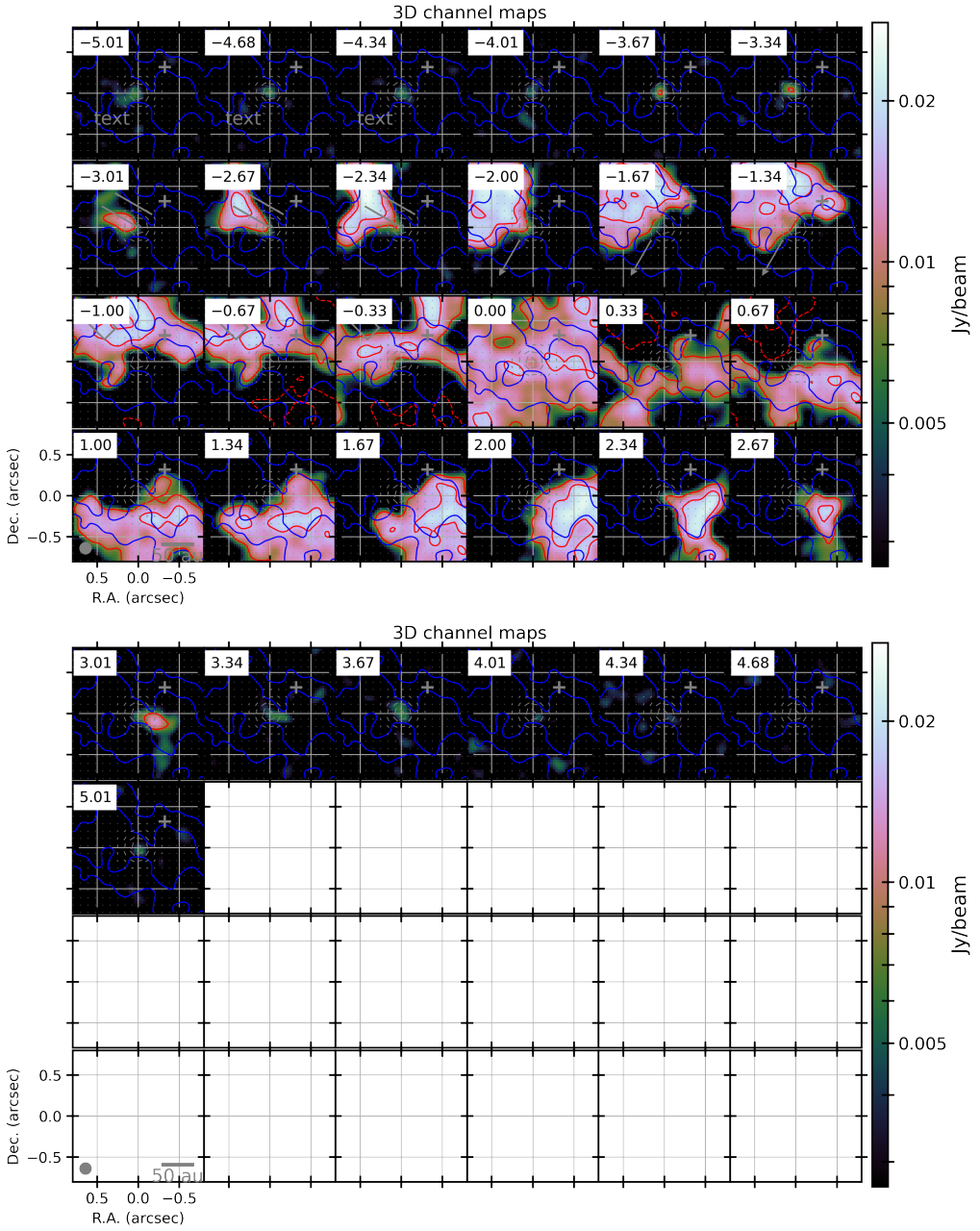


Figure 4: Example of channel maps. In addition to the color maps, scale bars, and beams, this figure shows contour maps, segment maps, regions, markers, lines, arrows, and text. The number at the top right corner at each channel denotes the velocity in the unit km s^{-1}. The upper and lower panels are in separated image files.

6.7 Animation

The PlotAstroData class can be used together with `matplotlib.animation` to create an animation file from a fits file. The following is an example using a fits file in the GitHub site of `plotastrodata`.

```
import numpy as np
import matplotlib.pyplot as plt
import matplotlib.animation as animation
from plotastrodata.plot_utils import PlotAstroData


def update_plot(i):
    # Clear the previous image and colorbar axes.
    fig.clear()
    ax = fig.add_subplot(111)
    f = PlotAstroData(rmax=0.8, vmin=-5, vmax=5,
                      fitsimage='example_data/test3D.fits',
                      channelnumber=i, fig=fig, ax=ax)
    f.add_color(fitsimage='example_data/test3D.fits')
    f.set_axis()
    f.fig.tight_layout()

nchans = 61
fig = plt.figure()
ani = animation.FuncAnimation(fig, update_plot, frames=nchans, interval=50)
Writer = animation.writers['ffmpeg']
writer = Writer(fps=10, bitrate=128)
ani.save('test_animation.mp4', writer=writer)
plt.close()
```

For repeated animation frames, clearing the whole figure removes both the previous image and its colorbar axes. Supply the new axes together with the same figure on every call.

6.8 Other plotting functions

The function `plot3d` is a function independent of the PlotAstroData class. This function provides an HTML file to show a 3D figure that can be interactively rotated in a web browser.

```
mom0 = np.sum(d.data, axis=0) * d.dv
sigma_mom0 = d.sigma * abs(d.dv) * np.sqrt(len(d.v))
vplus = {'data': mom0, 'sigma': sigma_mom0, 'cmap': 'gray'}
plot3d(**d.todict(),
       levels=[3, 6, 9],
       cmap='Jet',
       alpha=0.08,
       vplus=vplus,
       xlabel='R.A. (arcsec)',
       ylabel='Dec. (arcsec)',
       vlabel='Velocity (km/s)',
       eye_p=0,
       eye_i=180,
       outname='filename',
       show=True)
```

`plot3d` requires `dist=1`, `swapxy=False`, and one non-PV cube with at least two samples on each uniformly spaced coordinate axis. Coordinates are ascending to retain a right-handed system;

`xflip` and `yflip` do not reverse them. A finite positive scalar noise level is required after noise estimation. Nonfinite voxels are replaced by zero in a private plotting copy. RGB and 3D coordinate grids use the first interval as their uniform-spacing reference, with `rtol=1e-5, atol=0`; descending coordinates are allowed. RGB maps likewise require compatible uniform spatial grids and use lower origin with pixel-edge extents to align with color and contour maps.

The data to be plotted can be an instance of AstroData. `levels` specifies which surfaces are plotted. `cmap` is an argument (named colorscale) used in the 'mesh3d' type data in plotly.graph_objs. `alpha` is also used through the 'mesh3d' type data in plotly.graph_objs. `eye_p` and `eye_i` specify the viewing direction when the file is opened. The output has the extension of .html with the file name of `outname`. The argument `show=True` opens the exported HTML in a browser. Figure 5 shows an example with the test fits file in the GitHub site of `plotastrodata`.

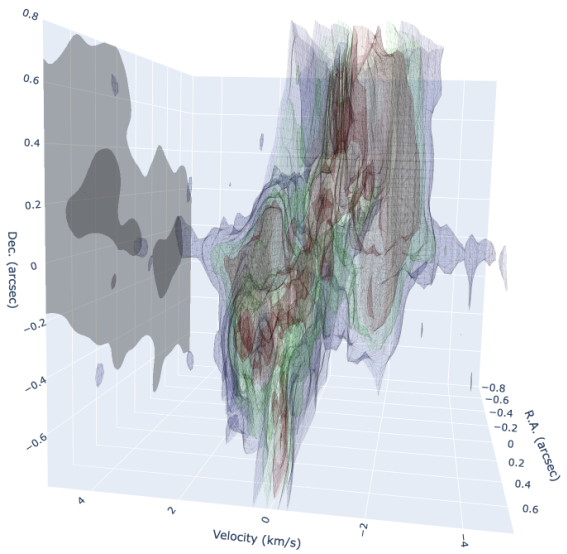


Figure 5: Snapshot of the html file on a web browser, made by `plot_utils.plot3d`. The actual html file can be rotated freely on a web browser.

The `plot_utils` module has more functions to make other types of figures. `plotprofile` provides a figure of line profiles and `plotslice` provides a 1D slice of a 2D image. These two functions use `AstroData.profile` and `AstroData.slice`, respectively. Hence, it may be more flexible to make figures in one's preferred way using the outputs of these method than using `plotprofile` and `plotslice`.

Appendix

A Coordinate Transformations

The relation between relative and absolute coordinates is not trivial on the plane of the sky. For example, the point 30 degrees (= 2h) east of '00h00m00s 60d00m00s' has coordinates of '03h16m25.6s 48d35m25s', rather than '2h00m00s 60d00m00s'. The module `coord_utils` provides useful functions for the coordinate transformation.

```
from plotastrodata.coord_utils import xy2coord
from plotastrodata.coord_utils import coord2xy
s = xy2coord(xy=[[30, 90], [0, 0]], coordorg='00h00m00s 60d00m00s')
print(s)
#['03h16m25.58528421s +48d35m25.36040662s', '06h00m00s +00d00m00s']
a = coord2xy(coords=s, coordorg='00h00m00s 60d00m00s')
print(a)
#[[30, 90], [0, 0]]
#[x_array, y_array]
```

This example calculates 30 degrees east (and 0 degrees north) and 90 degrees east (and 0 degrees north) of '00h00m00s 60d00m00s' and puts back the resulting coordinates to the relative coordinates in the unit of degree ([offset to the east, offset to the north]). The coordinates may include a frame, like 'ICRS 00h00m00s 60d00m00s' and 'B1950 00h00m00s 60d00m00s'.

B Line-of-Sight Rotation

Comparison of a physical model and an observational result often requires 3D rotation between the model coordinates and the observational coordinates. The `los_utils` module provides a useful coordinate transformation. It starts with a parabolic streamer, as an example, on a coordinate set, $(x_{\rm sys}, y_{\rm sys}, z_{\rm sys})$, named system coordinates, as shown in Figure 6(a). The direction of such a streamer is defined with polar and azimuthal angles (θ_0 and ϕ_0, respectively) in a 3D coordinate, $(x_{\rm phy}, y_{\rm phy}, z_{\rm phy})$, named physical coordinates. Figures 6(b) and 6(c) show the relation between $(x_{\rm sys}, y_{\rm sys}, z_{\rm sys})$ and $(x_{\rm phy}, y_{\rm phy}, z_{\rm phy})$, through an intermediate coordinate set, $(x_{\rm azi}, y_{\rm azi}, z_{\rm azi})$, named azimuthal coordinates.

When a model in the physical coordinates is observed, the reltaion beteen the physical and observational coordinates $(x_{\rm obs}, y_{\rm obs}, z_{\rm obs})$ requires inclination and position (or orientation) angles. Although this relation also requires another azimuthal angle, the ϕ_0 devined above will work for the azimuthal rotation. Figures 7(a) and 7(b) show the relation between $(x_{\rm phy}, y_{\rm phy}, z_{\rm phy})$ and $(x_{\rm obs}, y_{\rm obs}, z_{\rm obs})$, through an intermediate coordinate set, $(x_{\rm ori}, y_{\rm ori}, z_{\rm ori})$, named oriented coordinates. In this definition, when the zero position angle (P.A.= 0) means a configuration where the $z_{\rm phy}$ direction is projected to the $y_{\rm obs}$ direction. In other words, *this P.A. is not that of the disk major axis* but that of the blueshifted outflow in a typical protostellar system.

Figure 8 summarizes the relation between the system and observational coordinates with the four angles (θ_0, ϕ_0, incl., and P.A.) through the physical coordinates. The relation can be expressed

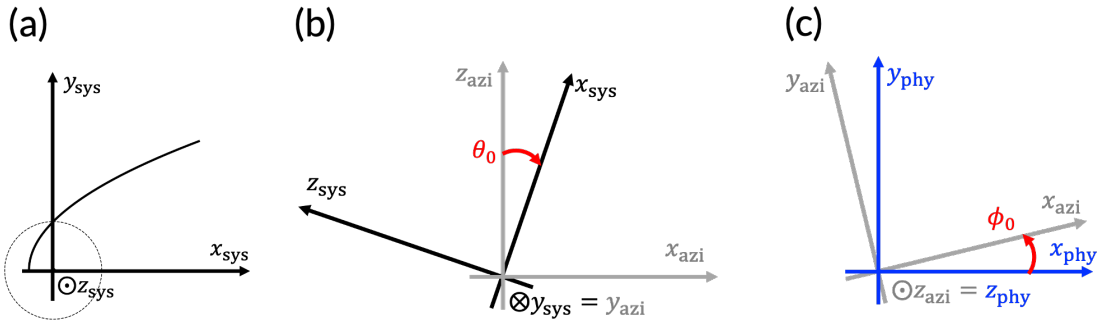


Figure 6: (a) The streamer system. (b) The system and azimuthal coordinates. (c) The azimuthal and physical coordinates.

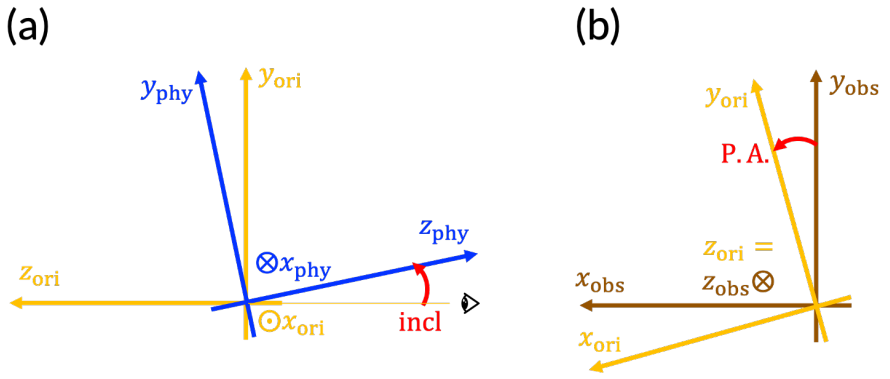


Figure 7: (a) The physical and oriented coordinates. (b) The oriented and observational coordinates.

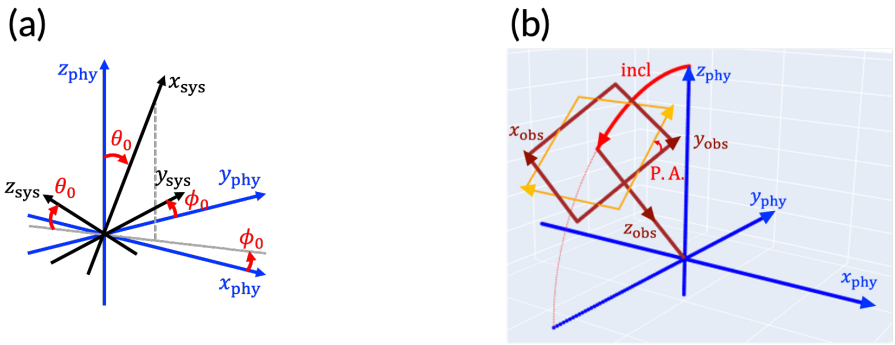


Figure 8: (a) The system and physical coordinates. (b) The physical and observational coordinates.

by the rotational matrices $R_x(\theta)$, $R_y(\theta)$, and $R_z(\theta)$ as follows:

$$\boldsymbol{x}_{\rm sys} = R_y(\frac{\pi}{2} - \theta_0)\boldsymbol{x}_{\rm azi}, \tag{14}$$

$$\boldsymbol{x}_{\rm ori} = R_z(-\phi_0)\boldsymbol{x}_{\rm phy}, \tag{15}$$

$$\boldsymbol{x}_{\rm phy} = \begin{pmatrix} -1 & 0 & 0 \\ 0 & 1 & 0 \\ 0 & 0 & -1 \end{pmatrix} R_x(-{\rm incl.})\boldsymbol{x}_{\rm ori}, \tag{16}$$

$$\boldsymbol{x}_{\rm ori} = R_z({\rm P.A.})\boldsymbol{x}_{\rm obs}. \tag{17}$$

The transformation between $\boldsymbol{x}_{\rm obs}$ and $\boldsymbol{x}_{\rm sys}$ can be done through the functions `obs2sys` and `sys2obs` in `los_utils`.

```
from plotastrodata.los_utils import obs2sys
from plotastrodata.los_utils import sys2obs
import numpy as np

xobs, yobs, zobs = sys2obs(xsys=1/np.sqrt(2), ysys=1/np.sqrt(2), zsys=0,
pa=0, incl=30, phi0=0, theta0=90, polar=False)
print(xobs, yobs, zobs)
#-0.707107 0.612372 0.353553
#-1/sqrt(2) cos(30deg)/sqrt(2) sin(30deg)/sqrt(2)
xsys, ysys, zsys = obs2sys(xobs=xobs, yobs=yobs, zobs=zobs, pa=0, incl=30,
phi0=0, theta0=90, polar=False)
print(xsys, ysys, zsys)
#0.7071077 0.707107 0.000000
#1/sqrt(2) 1/sqrt(2) 0
```

The input coordinate (e.g., `xsys` or `xobs`) may be a numpy array. When `polar=True`, `xsys`, `ysys`, and `zsys` are the radius, polar angle, and azimuthal angle, respectively; the radius is in the unit of arcsec, and the polar and azimuthal angles are in the unit of radian. `xobs`, `yobs`, and `zobs` are the Cartesian coordinates regardless of the `polar` argument. In particular, the combination of `polar=True` and $\theta_0 = 90$ is useful when a model is defined in the physical coordinates in the polar format.

In addition to the functions for the spatial coordinate transformation, `los_utils` has another function, `polarvel2losvel`, to transform polar velocities (v_r, v_θ, v_ϕ) to line-of-sight velocity (i.e., $v_{z,{\rm obs}}$).

```
from plotastrodata.los_utils import polarvel2losvel
import numpy as np

v = polarvel2losvel(v_r=-1/np.sqrt(2), v_theta=0, v_phi=1/np.sqrt(2),
theta=np.pi/2, phi=np.pi/2, incl=30, phi0=0, theta0=90)
print(v)
#-0.353553
#-1/sqrt(2)/2
```

`theta` and `phi` may be numpy arrays and in the unit of radian. `incl`, `phi0`, and `theta0` are the same as in `obs2sys` and `sys2obs`. This velocity transformation is useful for calculating the line-of-sight velocity when the 3D velocity field of a model is written in the polar format as a function of the polar coordinates. This velocity transformation can be derived from the following ideas. First,

a vector is expressed as a matrix product of a basis matrix and a component column vector.

$$E_1\boldsymbol{x}_1 := \begin{pmatrix}\boldsymbol{e}_{x,1} & \boldsymbol{e}_{y,1} & \boldsymbol{e}_{z,1}\end{pmatrix}\begin{pmatrix}x_1\\y_1\\z_1\end{pmatrix} = \begin{pmatrix}\boldsymbol{e}_{x,2} & \boldsymbol{e}_{y,2} & \boldsymbol{e}_{z,2}\end{pmatrix}\begin{pmatrix}x_2\\y_2\\z_2\end{pmatrix} =: E_2\boldsymbol{x}_2 \tag{18}$$

$$\boldsymbol{x}_1 = E_1^{-1}E_2\boldsymbol{x}_2 \tag{19}$$

This provides the change-of-basis matrix. This matrix can also change the basis for velocity vectors.

$$\boldsymbol{v}_1 = E_1^{-1}E_2\boldsymbol{v}_2 \tag{20}$$

Hence, we already know the relation between $\boldsymbol{v}_{\rm ori}$ and $\boldsymbol{v}_{\rm sys}$.

$$\boldsymbol{v}_{\rm ori} = R_x(\text{incl.})\begin{pmatrix}-1 & 0 & 0\\0 & 1 & 0\\0 & 0 & -1\end{pmatrix}R_z(\phi_0)R_y(\theta_0-\frac{\pi}{2})\boldsymbol{v}_{\rm sys} \tag{21}$$

The polar velocities can be expressed with the Cartesian velocities as follows:

$$\begin{pmatrix}v_r\\v_\theta\\v_\phi\end{pmatrix} = \begin{pmatrix}\sin\theta\cos\phi & \sin\theta\sin\phi & \cos\theta\\\cos\theta\cos\phi & \cos\theta\sin\phi & -\sin\theta\\-\sin\phi & \cos\phi & 0\end{pmatrix}\begin{pmatrix}v_x\\v_y\\v_z\end{pmatrix} =: E_r^{-1}\boldsymbol{v}. \tag{22}$$

Then, the polar velocities are linked to $\boldsymbol{v}_{\rm sys}$.

$$\boldsymbol{v}_{\rm sys} = E_r\begin{pmatrix}v_{r,\rm sys}\\v_{\theta,\rm sys}\\v_{\phi,\rm sys}\end{pmatrix} = (E_r^{-1})^T\begin{pmatrix}v_{r,\rm sys}\\v_{\theta,\rm sys}\\v_{\phi,\rm sys}\end{pmatrix} \tag{23}$$

Finally, the line-of-sight velocity is calculated as the z component of $\boldsymbol{v}_{\rm z,ori}$ because the rotation about P.A. does not change the line-of-sight velocity.

$$v_{\rm los} = v_{z,\rm obs} = v_{z,\rm ori} \tag{24}$$

C Fourier Transform

The Fourier transform frequently appears in the context of (radio) astronomy. Although it can be easily done by a computer, the numerical way of calculating the Fourier transform is different from the mathematical way. The main difference is the phase center. The `fft_utils` module provides an output closer to the mathematical way. This module is a wrapper of `numpy.fft`. The following shows a comparison between the `plotastrodata.fftcentering` module and the original `numpy.fft` module.

```
# Example of numpy.fft.fft
import numpy as np

x = np.linspace(-99.5, 99.5, 200)
f = np.where(np.abs(x) < 10, 1, 0)

u = np.fft.fftshift(np.fft.fftfreq(len(x), d=x[1]-x[0]))
F = np.fft.fftshift(np.fft.fft(f))
```

```
# Example of plotastrodata.fft_utils
from plotastrodata.fft_utils import fftcentering
import numpy as np

x = np.linspace(-99.5, 99.5, 200)
f = np.where(np.abs(x) < 10, 1, 0)

F, u = fftcentering(f=f, x=x, xcenter=0)
```

Figure 9 shows the input and output functions obtained by the example scripts above using `numpy.fft.fft` and `plotastrodata.fft_utils.fftcentering`. One will expect that the phase center is at $x = 0$ or the center of x, and then the expected Fourier transform of Figure 9(a) will be Figure 9(c). The phase center is indeed specified in `fftcentering` by the argument of `xcenter=0`. In comparison, Figure 9(b) (i.e., the output of `numpy.fft.fft` alone) can be produced by `xcenter=x[0]`.

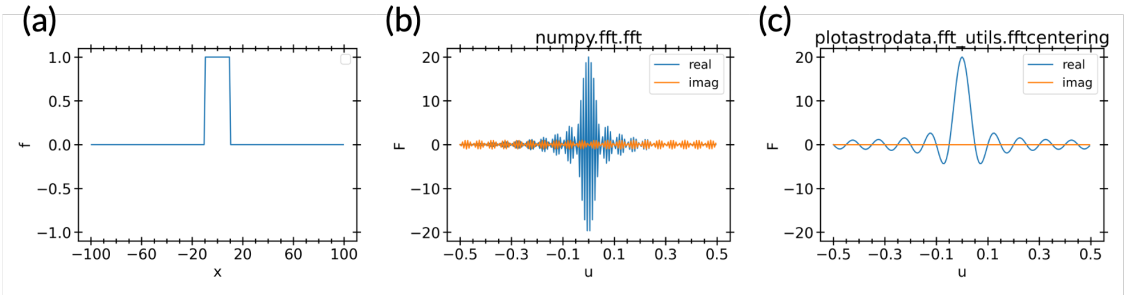


Figure 9: (a) The input function for the Fourier transform. (b) The real and imaginary parts obtained by `numpy.fft.fft`. (c) The real and imaginary parts obtained by from `plotastrodata.fft_utils.fftcentering`.

The inverse Fourier transform can be done by `ifftcentering` as follows.

```
from plotastrodata.fft_utils import fftcentering
from plotastrodata.fft_utils import ifftcentering
import numpy as np

x = np.linspace(-99.5, 99.5, 200)
f = np.where(np.abs(x) < 10, 1, 0)

F, u = fftcentering(f=f, x=x, xcenter=0)
ynew, xnew = ifftcentering(F=F, u=u, xcenter=0, x0=x[0], outreal=True)
print(np.max(np.abs(xnew - x)) < 1e-10, np.max(np.abs(ynew - f)) < 1e-10)
#True, True
```

The last two outputs are the same as the two inputs within the numerical error, which is consistent with iFT[FT[f]] = f. `ifftcentering` also requires the phase center value and additionally the starting value of the iFTed coordinate (xnew in the above example). This is because the Fourier transform is independent of absorlute values of the input coordinate. The argument of `outreal=True` means that the output ynew has only real values.

It is also possible for `fft_utils` to shift the phase of given FTed function and coordinate, by `fft_utils.shiftphase`.

```
from plotastrodata.fft_utils import shiftphase
import numpy as np

x = np.linspace(-99.5, 99.5, 200)
f = np.where(np.abs(x) < 10, 1, 0)

u = np.fft.fftshift(np.fft.fftfreq(len(x), d=x[1]-x[0]))
F = np.fft.fftshift(np.fft.fft(f))
F = shiftphase(F=F, u=u, xoff=-x[0])
```

The output of `shiftphase` is the same as the first output (F in the example) obtained by `fftcentering`. The offset argument of `xoff` is determined by "new center (0) − old center (x[0])".

The functions introduced above (`fftcentering`, `ifftcentering`, and `shiftphase`) have counterparts for 2D arrays: `fftcentering2`, `ifftcentering2`, and `shiftphase2`. `fftcentering2` requires an argument of `y`, as well as `x`; these two arguments can be either 1D or 2D arrays (after `numpy.meshgrid`). The center is also expressed by the two arguments `xcenter` and `ycenter`. Similarly, `ifftcentering2` takes arguments of `u`, `v`, `xcenter`, `ycenter`, `x0`, and `y0`, and `shiftphase2` takes arguments of `u`, `v`, `xoff`, and `yoff`.

For real-input FFTs (`rfft=True`), the half-spectrum does not uniquely determine the original x-axis sample count. Pass the original count using the keyword-only `nx` argument of `ifftcentering` or `ifftcentering2` to distinguish odd and even lengths. When relying on inference instead, supply the original spatial spacing `dx`, even if frequency coordinates are supplied.

D Fitting

Data analysis may lead a fit with a model to the data. For this purpose, the `plotastrodata` package has a module of `fitting_utils`, a wrapper of `emcee` version 3.0 or later (and `ptemcee`), `corner`, and `dynesty`; all of these packages are widely used in astronomical research for MCMC (Markov chain Monte Carlo) fitting.

Fitting process usually maximize a log-likelihood function. Here, a very simple function is used as the log-likelihood function with three variables.

```
def logl(p):
    x1, x2, x3 = p
    chi2 = (x1 / 1)**2 + (x2 / 2)**2 + (x3 / 4)**2
    return -0.5 * chi2
```

Then, the class of EmceeCorner in `fitting_utils` needs to be instantiated with a list of parameter boundaries (`bounds`) as well as the log-likelihood function.

```
from plotastrodata.fitting_utils import EmceeCorner

fitter = EmceeCorner(bounds=[[-5, 5], [-10, 10], [-20, 20]],
                     logl=logl, progressbar=False, percent=[16, 84])
```

If `progressbar=True`, some fitting methods show a progress bar in the terminal. `percent` specifies which percentiles are used to evaluate the uncertainty of each parameter. The fitting process is mainly performed by the `fit` method with arguments of `nwalkersperdim` (number of walkers

per parameter), `nsteps` (number of steps including the burned-in steps), and `nburnin` (number of steps to be burned-in). The argument `ncores` specifies the number of processes used by `multiprocessing.Pool`; `ncores=1` performs serial sampling. When `ncores>1`, user-supplied `logl` and `model` functions must be pickleable, for example by defining them at module scope rather than inside another function. Serial and parallel `emcee` sampling apply the same bounded uniform prior specified by `bounds`.

```
fitter.fit(nwalkersperdim=30, nsteps=11000, nburnin=1000,
           ntemps=1, savechain='chain.npy')
print('best:', fitter.popt)
#best: [-0.008 -0.026  0.008]
print('lower percentile:', fitter.plow)
#lower percentile: [-0.988 -1.954 -3.976 ]
print('50 percentile:', fitter.pmid)
#50 percentile: [ 0.007  0.030 -0.020]
print('higher percentile:', fitter.phigh)
#higher percentile: [0.990 2.012 3.955]
```

When `ntemps>1`, the `fit` method uses `ptemcee`, rather than `emcee`, to incorporate multiple temperatures. `savechain` specifies a file name to which the obtained post-burn-in chain is saved; if this is None, the chain is not saved to any file. Both the saved chain and the `samples` attribute have the shape `(steps, walkers, parameters)`.

When two fitting results of the same data are compared between different models, a quantity of "evidence" can be used. The evidence is defined as a fractional volume of the likelihood function L in the parameter $(p_1 \cdots p_n)$ space: $\int \cdots \int L dp_1 \cdots dp_n / \int \cdots \int dp_1 \cdots dp_n$. Because the integrated range is not from infinity to infinity, the evidence value should be slightly smaller than $(\sqrt{2\pi} \cdot 2\sqrt{2\pi} \cdot 4\sqrt{2\pi})/(10 \cdot 20 \cdot 40) = 0.0157$ in the example. The evidence can be calculated using the method of `getDNSevidence`. This method uses DynamicNestedSampler in `dynesty`. The arguments for DynamicNestedSampler can also be used for this method.

```
fitter.getDNSevidence()
print('evidence:', fitter.evidence)
#evidence: 0.0153
```

The result of MCMC fitting can be plotted in two ways. One is the so-called corner plot, which shows the likelihood function as a function of each parameter or each pair of parameters. This can be done through the method of `plotcorner`. `show` specifies whether to show the figure in a window. `savefig` specifies the file name to which the figure is saved. `labels` specifies the parameter names that will be shown in the figure. `cornerrange` is a list of the boundaries for the figures or a list of fractions to be plotted.

```
fitter.plotcorner(show=True, savefig='corner.png',
                  labels=['par1', 'par2', 'par3'],
                  cornerrange=[[-4, 4], [-8, 8], [-16, 16]])
```

Another way is plotting the chains in the MCMC run as a function of the step. This can be done through the method of `plotchain`. The arguments have the same meaning as those in `plotcorner`.

```
fitter.plotchain(show=True, savefig='chain.png',
                 labels=['par1', 'par2', 'par3'],
                 ylim=[[-2, 2], [-4, 4], [-8, 8]])
```

The figure obtained through `plotchain` is simplified from the original chain. First, this method calculates the two percentiles given in the instantiation and the 50th percentile in the chain direction at each step. The two given percentiles are shown in a fainter color, while the 50th percentile is shown in a darker color. In addition, this method calculates the three percentiles in the step direction for each of the three percentiles derived above. For example, if 16th and 84th percentiles are given, this method calculates 16th percentile of 16th percentile, 50th percentile of 16th percentile, 84 percentile of 16th percentile, 16th percentile of 50th percentile, ..., and 84 percentile of 84 percentile. The second calculation in the step direction uses 1% of the total number of steps; if the total number of steps is < 200, the second calculation is skipped. In the figure, the 50th percentile of each percentile is shown with a thicker line, while the other two percentiles of each percentile is shown with a thinner line. As a result, the figure shows one thick-dark, two thick-faint, two thin-dark, and four thin-faint lines. Figures 10(a) and 10(b) show the outputs of `plotcorner` and `plotchain`, respectively.

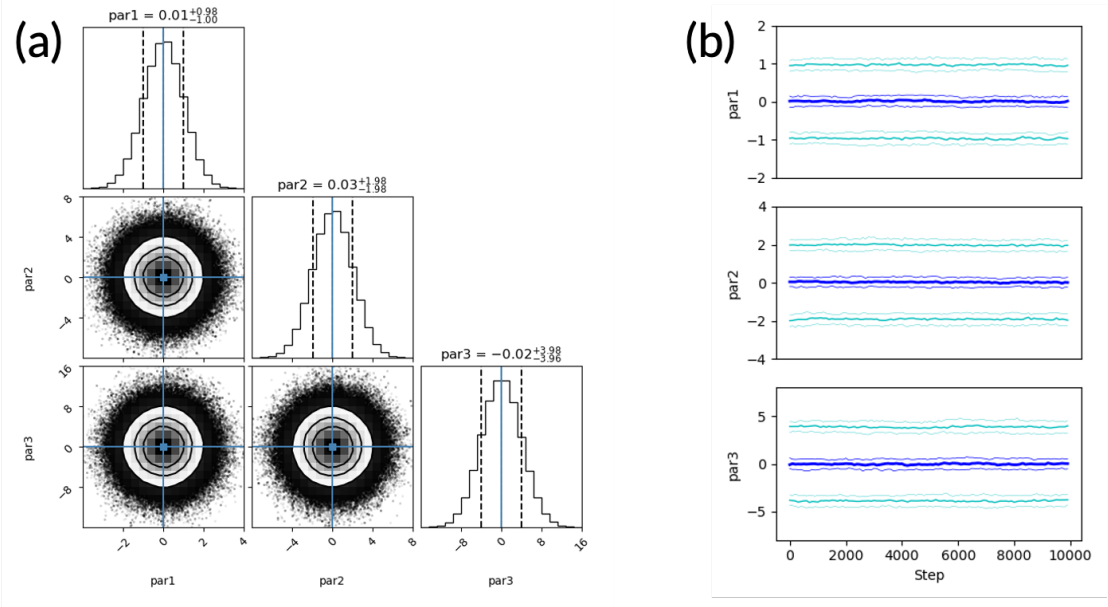


Figure 10: The results of (a) `plotcorner` and (b) `plotchain` in the example.

`plotcorner` also supports a one-parameter fit, producing a single marginal distribution. The standalone `gaussfit1d` and `gaussfit2d` functions default to `sigma=None`: they print a notice and estimate noise from residuals of an initial fit. Both functions accept `progressbar=False` by default; set it to True to display progress for each pass. Both functions always perform two fitting passes, refining the parameter bounds around the first result for the second pass, even when sigma is supplied. The 1D fitter requires at least two samples and nonzero coordinate and data ranges. NaN and infinite values in the 1D fitting coordinates or samples raise ValueError; the 2D fitter likewise rejects nonfinite image pixels. Returned scalar sigma is a Python float, preserving extended precision; array-valued sigma retains its dtype.

D.1 Grid Search

When the number of parameters in a fit is not many, it might be an option to calculate the log-likelihood function simply on a parameter grid, without the MCMC technique. This simple calculation also helps to check the overall shape of the log-likelihood function independently from the number of walkers or the number of steps of the MCMC run. This grid search can be performed using the method of `posteriorongrid` with an argument of `ngrid`, which specifies the number of grids for each parameter.

```
fitter.posteriorongrid(ngrid=[101, 201, 401])
print('best:', fitter.popt)
#best: [0. 0. 0.]
print('lower percentile:', fitter.plow)
#lower percentile: [-1. -2. -4.]
print('50th percentile:', fitter.pmid)
#50th percentile: [0. 0. 0.]
print('higher percentile:', fitter.phigh)
#higher percentile: [1. 2. 4.]
print('evidence:', fitter.evidence)
#evidence: 0.0155
```

The `posteriorongrid` method directly substitutes the meshed grid of the parameters to the log-likelihood function: for example, `np.meshgrid(p3, p2, p1, indexing='ij')` for three parameters. For this reason, the log-likelihood function must separate the parameter grid and the data grid. For example, if the log-likelihood function includes a subtruction of `x - p2`, where `x` is a data grid, this must be expressed as `np.subtract.outer(x, p2)` to separate the dimensions for the parameters from the dimension(s) for the data.

The result of the grid search can be visualized, as a corner plot, by the method of `plotongrid`. The arguments are the same as for `plotcorner`.

```
fitter.plotongrid(show=True, savefig='grid.png',
                  labels=['par1', 'par2', 'par3'],
                  cornerrange=[[-4, 4], [-8, 8], [-16, 16]])
```

Figure 11 shows the corner plot output from `plotongrid`. The first parameter has the coarsest grid compared to the width of the log-likelihood function.

E Additional Utilities

The `plotastrodata` package has a bit more things useful for astronomical research. Even though they might not be worth using from this package, it could make your code simpler to import them from the same package when other modules are imported from `plotastrodata`.

`const_utils` is a wrapper of `astropy.constants` and `astropy.units`. The variables defined in `const_utils` are Python numerical scalars in SI units. These values are obtained through the `.to('unit name')` method in `astropy.units` and the `.value` attribute in `astropy.constants`. In addition, `const_utils` has variables with the name of metric prefixes from quecto (10^{-30}) to quetta (10^{30}). These variables are Python scalars; integer constants retain integer type.

```
import plotastrodata.const_utils as cu
print(cu.pc)
#3.085677581491367e+16
```

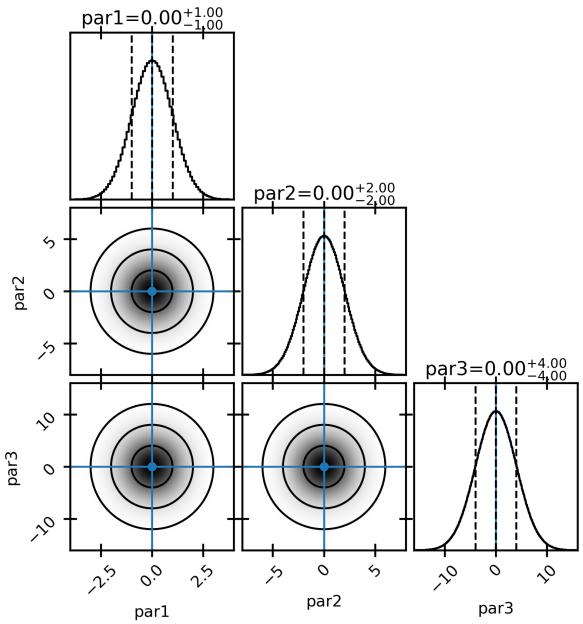


Figure 11: The results of `plotongrid`.

```
print(cu.M_sun)
#1.988409870698051e+30
print(cu.centi)
#0.01
```

`ext_utils` provides functions that are often used in astronimcal research. The Planck functions accept scalar inputs and N-D NumPy arrays.

```
from plotastrodata.ext_utils import BnuT
# Planck function. 30 K, 230e9 Hz.
x = BnuT(T=30, nu=230e9)
print(x)
#4.033705316844142e-16
```

```
from plotastrodata.ext_utils import JnuT
# Brightness temperature (=wavelenth^2 / (2k_B) * BnuT). 30 K, 230e9 Hz.
x = JnuT(T=30, nu=230e9)
print(x)
#24.818562479617647
```

`costaper(shape, npix=0)` returns a float64 cosine window with the requested 1D or 2D shape. Pass a tuple of one or two nonnegative integer axis lengths, such as `(nx,)` or `(ny, nx)`; boolean lengths are rejected. To taper an array, multiply it by `costaper(data.shape, npix)`. Each axis has a half-cosine ramp over `npix` pixels at both edges, with zero at the outermost pixel and unit weights in the interior. Two-dimensional windows are outer products of the axis windows. The integer width must be nonnegative and at most half the length of every axis; booleans and overlapping tapers are rejected. The default width is zero, which returns all ones (no taper).

```
from plotastrodata.ext_utils import costaper
window = costaper((6,), npix=2)
# [0., 0.5, 1., 1., 0.5, 0.]
```

F Estimating Plotting Ranges

After reading an AstroData instance with AstroFrame, use `rmax, vmin, vmax = d.estimate_ranges(pbcor=True)` for PB-corrected data. For uniform-noise data, omit `pbcor=True`. The stored `d.sigma` must be a finite positive scalar or `None` to estimate noise. With a supplied sigma and `pbcor=False`, assume a zero baseline. For PB-corrected data, a supplied sigma describes noise before PB correction; fitting is still needed for PB preparation. The method leaves data and stored attributes unchanged. Images return `None` for both velocity limits. Spatial limits use the coordinate units; velocity limits use the units of `v` and are returned in ascending order. `rmax` is a symmetric spatial half-width. Numeric results are Python floats, except `np.longdouble` results retain extended precision. The `pbcor` argument accepts a Python bool.

The array-based interface is `noise_utils.estimate_ranges(data, x, y, sigma=None, v=v, beam=beam, pbcor=False)`. Coordinates must be uniformly spaced; omit `v` for an image. The beam is optional; if either FWHM length is `None`, pixel-based defaults are used. Script generators use `AstroData.estimate_ranges(pbcor=True)` with the stored `sigma=None` and write the estimated limits as editable numeric values.

G Simple 2D Script Generation

The `script_utils.generate_image_script` function writes an editable script using the existing AstroData, AstroFrame, Noise, and PlotAstroData APIs. Noise and range estimation assume PB-corrected intensities. It generates a noise visualization, a 2D Gaussian fit reporting the sky center and east/north uncertainties in arcsec, and an image with a logarithmic color map, default contours and beam, and RA/Dec axes.

```
from plotastrodata.script_utils import generate_image_script
script = generate_image_script("image.fits", output="image.py")
# Review image.py, then run: python image.py
```

The input must be a primary-HDU RA/Dec image with at least two pixels on each spatial axis; higher axes must be singleton. Generation reads the header and pixels to estimate the crop, but does not execute the script. The function accepts an input FITS path, an optional output script path, and the keyword-only boolean `overwrite`. The output must have a `.py` suffix; its parent directories are created when needed, and the generated source is checked for Python syntax before writing. The return value is the absolute script path as a pathlib.Path. Existing scripts require `overwrite=True` to replace them.

The generator estimates rmax and writes it directly into the fitting and plotting frames as an inline number rounded to three significant digits. Edit these values in the script to change either region. Generation reads the input and fits noise to estimate ranges but does not execute the generated script or create its figures.

The generated script reads the full image using AstroFrame.read with `sigma=None`, then fits and plots the Noise histogram with `sigma='hist-pbcor'`. Noise mean and standard deviation print with `.3e` format. The fitted standard deviation is assigned to `d.sigma`; an inline comment notes that a float in the data unit is also accepted. A second image read uses the fitted noise and smaller region for `gaussfit2d(progressbar=True)`. The Gaussian fitter applies beam noise correlation correction and rejects nonfinite pixels. It uses MCMC, which can take substantial time for large images.

The 2D image is centered on the sky position returned by the Gaussian fit. The script saves two PNGs beside itself: a noise histogram named `<script stem>_noise.png` and an image named `<script stem>.png`. Figure overwriting follows the plotting methods, independently of the generator's overwrite option. Plotting writes `stretch='log'` and `show=False` directly, with default fonts and no brightness-temperature conversion. Edit the generated calls to change these choices. No annotation overlays, automatic source selection, or helper functions are generated.

H Cube Script Generation

Noise and range estimation assume PB-corrected intensities. The `script_utils.generate_cube_script` function writes a noise visualization, an integrated line profile, 3D channel maps, and two moment-map figures. The profile and channel maps use the existing APIs from example.py.

```
from plotastrodata.script_utils import generate_cube_script
script = generate_cube_script("cube.fits", output="cube.py")
# Review cube.py, then run: python cube.py
```

Generation reads the primary FITS header and cube pixels. The input must have RA/Dec axes with at least two pixels each, at least two frequency or velocity channels on axis 3 (Hz, m/s, or km/s), and only singleton higher axes. The generated script reads the full cube using AstroFrame.read with `sigma=None`, fits Noise with `sigma='edge,hist-pbcor'`, saves the histogram as `<script stem>_noise.png`, and prints mean and standard deviation with `.3e` formatting. The first and last channels are selected; edit the selection if they contain emission. The fitted noise is reused for logarithmic colors and red contours.

Between noise visualization and channel maps, a separate AstroData instance shares the loaded cube and is read with AstroFrame using the estimated spatial and velocity ranges. The script calls plotprofile with `flux=True` and a circle of diameter four times rmax to enclose the entire cropped square. This assumes Jy/beam intensities and requires beam metadata and pixel sizes. The single-panel profile is saved as `<script stem>_profile.png` at intrinsic channel spacing. The shared crop can be edited independently, and the full cube remains available for channel mapping. The following excerpt uses illustrative ranges; the generator writes its estimated rmax, vmin, and vmax directly into both the crop frame and channel-map frame.

```
from plotastrodata.analysis_utils import AstroData, AstroFrame
from plotastrodata.plot_utils import plotprofile

d_crop = AstroData(**d.todict())
f_crop = AstroFrame(
    rmax=3.0,  # Estimated spatial half-width (arcsec).
    vmin=1.0, vmax=8.0,  # Estimated velocity range (km/s).
)
f_crop.read(d_crop)
plotprofile(**d_crop.todict(), xlist=[0], ylist=[0],
            ellipse=[4 * f_crop.rmax, 4 * f_crop.rmax, 0],  # Circle enclosing the
                cropped square.
            flux=True,
            savefig="cube_profile.png",
            show=False,
            )
```

PlotAstroData receives `v=d.v` and the cube center, followed by `set_axis()` and `savefig(..., show=False)`. The generator estimates rmax, vmin, and vmax and writes them into both frames as inline numbers rounded to three significant digits, with comments. Edit these values directly to change either region. Selected channels are plotted at intrinsic spacing without skipping or binning. Both plots show original PB-corrected data about the FITS reference center. Changing the script's noise fit does not recompute the fixed ranges. The noise histogram uses the full spatial footprint. The library supplies default panel layout, velocity labels, fonts, contours, and beam. No title or annotation overlays are added. Channel maps are saved beside the script as `<script stem>.png`, or `<script stem>_0.png`, `<script stem>_1.png`, and so on when multiple pages are needed. After the channel maps, the script calls `d_crop.moment(threshold_cube=3, threshold_mom0=3)`. The moments use the crop frame's spatial and velocity ranges. Edit that frame to change the ranges, or the two noise multiples to change the masks. The method supplies `mom0`, `mom1`, `mom2`, and `sigma_mom0`; the generated script uses these stored results directly. It then sets `d_crop.v=None` for 2D plotting. For each figure, `d_crop.data` is replaced with `mom0` and its sigma with `sigma_mom0` for contours; data is then replaced with `mom1` or `mom2` and sigma with `None` for colors. Both plotting calls use `**d_crop.todict()`. The final data is the 2D `mom2` array, and sigma and v are `None`. The moment arrays remain available on the object, and the original cube `d` is unchanged. The method

returns `None`; its stored `sigma_mom0` is a Python float, except `np.longdouble` retains extended precision. Array results keep their computed NumPy dtypes.

Two separate single-panel figures show moment 1 and moment 2 in linear colors, each overlaid with moment-0 contours at `[3, 6, 12, 24, 48, 96]` times `sigma_mom0`. Moment 1 uses the blue-red `RdBu_r` colormap; moment 2 uses the default colormap. Both figures have RA/Dec axes, default fonts and beam, `cbformat='%.1f'`, and `show=False`. They are saved beside the script as `<script stem>_mom1.png` and `<script stem>_mom2.png`. The moment-0 noise estimate is the channel noise times the absolute channel width times the square root of the number of selected channels. No separate error figure is generated.

The generator accepts an input FITS path, an optional output script path, and keyword-only `overwrite=False`, like the image generator. The return value is the absolute script path as a pathlib.Path, and generated input and figure paths are absolute. The output must end in `.py`. The generator checks Python syntax and creates parent directories before writing; it does not execute the script. Numeric estimates are written as Python float literals. Invalid headers, output suffixes, or finite crop regions raise ValueError; a non-boolean overwrite argument raises TypeError. Existing scripts require `overwrite=True`; figure overwriting follows the plotting methods independently. Edit choices directly in the script.